

Multi-label Twin support vector machine with probabilistic output and its safe screening rule

Bohan Zhang^a, Xincheng Lv^b, Chang Xu^a, Yitian Xu^{a,b}

^aCollege of Information and Electrical Engineering, China Agricultural University, Beijing 100083, China

^bCollege of Science, China Agricultural University, Beijing 100083, China

Abstract

Multi-label learning presents unique challenges in predicting and ranking multiple labels, especially in large-scale scenarios. Although Twin Support Vector Machine (TSVM) has demonstrated strong performance in binary classification tasks, their adaptation to the multi-label setting remains underexplored. Existing extensions typically assign a single hyperplane to each label and rely on manually defined thresholds for decision-making, which often leads to suboptimal results. In this work, we propose a novel Probabilistic Multi-Label Twin Support Vector Machine (PMLTSVM) framework that reintroduces twin hyperplanes for each label and integrates a probabilistic ranking strategy in place of conventional geometric ranking. This probabilistic perspective not only enhances robustness but also provides a more principled ranking of label relevance. To the best of our knowledge, it is the first attempt to combine probabilistic modeling with TSVM under the binary relevance paradigm. Furthermore, we develop a safe screening rule, capable of discarding non-influential training instances before optimization, thereby substantially reducing both training time and hyperparameter tuning efforts. The proposed method is supported by theoretical guarantees and validated through experiments, **consistently outperforming state-of-the-art methods by about 3%–10% in key metrics such as Hamming loss and Ranking loss, while achieving a maximum speedup of 49.3 on the datasets.**

Key words: Multi-label learning, Twin support vector machine, Probabilistic output, Safe Screening Rule.

1. Introduction

Supervised classification focuses on building mapping functions from input spaces to discrete label sets. Formally, given an input space $\mathcal{X} \subseteq \mathbb{R}^d$ and a finite label set $\mathcal{Y} = \{y_1, y_2, \dots, y_q\}$, a classification model $h : \mathcal{X} \rightarrow \mathcal{Y}$ learns sample-label relations from training data. When the label set contains two elements ($|\mathcal{Y}| = 2$), this becomes binary classification with single label assignments. For larger label sets ($|\mathcal{Y}| > 2$) under single-label constraints, it extends to multiclass classification. Multi-label classification removes these restrictions by allowing samples to be assigned to multiple label subsets $\mathcal{T}(x) \subseteq \mathcal{Y}$ and their complements $\mathcal{Y} \setminus \mathcal{T}(x)$. The prediction task in multi-label scenarios operates on two levels: (1) determining the presence/absence of each label through binary predictions (measured by Hamming Loss et al.); and (2) generating label ranking scores to evaluate model performance (assessed via Ranking Loss et al). This framework better matches the complexity of real-world data [1].

The practical importance of multi-label classification stems from its alignment with real-world data patterns. In social me-

dia image analysis, photographs often contain multiple semantic labels simultaneously such as “beach”, “sunset”, and “couple”. Electronic medical records frequently show symptom correlations, like diabetic retinopathy co-occurring with peripheral neuropathy. Gene expression analysis reveals that individual genetic loci can regulate multiple biological pathways. These real-world complexities establish multi-label learning as a crucial machine learning research area with demonstrated successes in text classification [2], medical diagnosis [3], and image classification [4].

Multi-label learning methodologies primarily adopt two technical paradigms [5]. The problem transformation paradigm converts multi-label tasks into established learning scenarios through three approaches: binary classification using the Binary Relevance (BR) [6] and its dynamic extension Classifier Chains (CC) [7]; label ranking systems exemplified by Calibrated Label Ranking (CLR) [8]; and multi-class conversion via the Random k-label sets [9]. Conversely, the algorithm adaptation paradigm modifies base learners to directly process multi-label data, including ML-kNN [10] adapting lazy learning, ML-DT [11] modifying decision trees, and CML [12] addressing label dependencies.

Due to the strong performance of SVMs in single-label clas-

¹ Corresponding authors. xytsuxue@126.com (Yitian Xu)

sification, there has been growing interest in extending them to multi-label learning. A classical method is Rank-SVM [13], which formulates label ranking as pairwise classification. ProSVM [14] later improved this by introducing the PRO loss. However, as pointed out in [15], the label distribution in multi-label learning is often highly imbalanced, and this class imbalance adversely affects the performance of Rank-SVM. Although weighting strategies [16] have been proposed, the issue remains. Moreover, Rank-SVM requires solving a large QPP with time complexity $O(l^3)$. To address these limitations, researchers turned to TSVM [17], which constructs two nonparallel hyperplanes optimized separately for positive and negative classes. During training, each hyperplane is optimized to be close to its own class samples and far from the opposite class samples. TSVM thus reduces the QPP size and training time while naturally avoiding the issue in traditional SVMs where a single hyperplane may be dominated by majority classes. **Due to its strong generalization performance, TSVM has been widely applied in various domains [18–20].**

Motivated by these properties and the suitability of TSVM to the data distribution in multi-label learning, Chen et al. [21] first extended TSVM to multi-label learning (MLTSVM). MLTSVM trains one hyperplane per label y_k and predicts based on the geometric distance $d_k = \frac{|w_k^T x + b_k|}{\|w_k\|}$ between the instance and each hyperplane. A label is assigned if $d_k < \Delta_k$, where Δ_k is an empirically chosen threshold. Label ranking is obtained by sorting the distances d_k . However, this approach has three limitations: (1) The threshold Δ_k lacks theoretical justification, (2) Only positive-class hyperplanes are trained, ignoring negative-class information, and (3) Directly ranking by d_k leads to suboptimal performance. To address these issues, we propose a probabilistic multi-label TSVM (PMLTSVM), which elegantly resolves all three limitations. For each label, we train both a positive-class hyperplane and a negative-class hyperplane by aggregating all other labels. The distance to the negative hyperplane d_{-k} replaces Δ_k . We then apply sigmoid calibration [22,23] to d_{+k} and d_{-k} to estimate the probability of label presence, leading to more robust prediction and improved label ranking.

Besides improving model performance, how to shorten the training time of SVM and its variants is also a closely watched direction. To increase the training speed, researchers have proposed many optimization algorithms. For example, Sequential Minimal Optimization (SMO) [24], Dual Coordinate Descent Method (DCDM) [25], Operator Splitting Solver for Quadratic Programs (osqp) [26]. Some researchers adopted methods for solving linear equations, such as the Successive Over-Relaxation (SOR) method [21], to solve the quadratic programming problem corresponding to SVM, and also achieved good results. These methods are trained using all the examples. In fact, SVM is a sparse model and only support vectors are used in the final decision. In a sparse model, many samples or features do not affect the final solution of the model. Therefore, Gao [27] developed Safe Sample Screening (SSR) to identify and remove inactive elements during training without performance loss. Wang et al. [28] enhanced this approach through variational inequality integration with SVM dual solutions (D-

VI), enabling pre-optimization sample filtering. Pan et al. designed safe screening rules for the parameter c of TSVM, and other screening rules were also proposed successively [29–31]. However, there are few applications of SSR in the multi-labeling domain. Existing SSR implementations mainly perform screening acceleration for RankSVM and ProSVM [32,34], but this method cannot be directly used in PMLTSVM. In fact, our proposed PMLTSVM is also a sparse model and can be safely accelerated. To our best knowledge, this is the first safe screening rule designed specifically for multi-label TSVM.

Inspired by the above studies, we propose a novel probabilistic multi-labeled dual support vector machine (PMLTSVM), which has three key advantages:

- (i) We re-substitute the single-threshold mechanism with paired hyperplanes $\{f_k^+(x), f_k^-(x)\}$ for relative distance comparisons, which solves the problem of hard-to-determine thresholds while making full use of the information from negative class samples.
- (ii) We use sigmoid function to convert geometric distances d_{-k}, d_{+k} into confidence probabilities, which gives the probability output of each label, and sort labels based on the probability output, which achieves performance improvement in sorting metrics such as Ranking Loss.
- (iii) A safe screening rule is proposed to accelerate training while maintaining prediction accuracy. Also, it can be combined with any existing optimization solver.

2. Related works

2.1. Twin support vector machine

TSVM [17] is originally proposed for binary single-label classification problem. Its essential idea is generating two non-parallel planes such that each plane is closer to one of the two classes and is as far as possible from the other.

$$f_1(x) = w_1^T x + b_1 = 0 \quad \text{and} \quad f_2(x) = w_2^T x + b_2 = 0, \quad (1)$$

where w_1, w_2 and b_1, b_2 are the normal vectors and bias terms of the above two hyperplanes, respectively.

Without loss of generality, we assume that the samples belong to n -dimensional real space $\mathbb{R}^n$. Let matrix $X_1 = (x_{11}, x_{12}, \dots, x_{1m_1})^T \in \mathbb{R}^{m_1 \times n}$ represent the training samples of class +1, and matrix $X_2 = (x_{21}, x_{22}, \dots, x_{2m_2})^T \in \mathbb{R}^{m_2 \times n}$ represent the training samples of class -1, where $m_1 + m_2 = m$.

To obtain the above two proximal hyperplanes, the optimization problems for TSVM can be formulated in matrix form as:

$$\begin{aligned} \min_{w_1, b_1, \xi_2} \quad & \frac{1}{2} \|X_1^T w_1 + e_{m_1} b_1\|^2 + c_1 e_{m_2}^T \xi_2 \\ \text{s.t.} \quad & -(X_2^T w_1 + e_{m_2} b_1) + \xi_2 \geq e_{m_2}, \quad \xi_2 \geq 0, \end{aligned} \quad (2)$$

$$\begin{aligned} \min_{w_2, b_2, \xi_1} \quad & \frac{1}{2} \|X_2^T w_2 + e_{m_2} b_2\|^2 + c_2 e_{m_1}^T \xi_1 \\ \text{s.t.} \quad & (X_1^T w_2 + e_{m_1} b_2) + \xi_1 \geq e_{m_1}, \quad \xi_1 \geq 0. \end{aligned} \quad (3)$$

where c_1 and c_2 are penalty parameters, and $\xi_1 = (\xi_{11}, \xi_{12}, \dots, \xi_{1m_1})^\top$ and $\xi_2 = (\xi_{21}, \xi_{22}, \dots, \xi_{2m_2})^\top$ are the slack variables. A new unseen instance $\mathbf{x}_t$ is assigned to label “+1” or “-1” depending on which of the proximal hyperplanes (1) it lies closer to.

2.2. TSVM probability model

In TSVM, the new sample will be assigned to either class +1 or -1 depending on its proximity to the two hyperplanes. Based on TSVM, the TSVM probability model[22] is established to describe the probability that a sample belongs to certain class. We consider two separating hyperplanes:

$$\frac{\mathbf{w}_1^\top \mathbf{x} + b_1}{\|\mathbf{w}_1\|} + \frac{\mathbf{w}_2^\top \mathbf{x} + b_2}{\|\mathbf{w}_2\|} = 0, \quad (4)$$

and

$$\frac{\mathbf{w}_1^\top \mathbf{x} + b_1}{\|\mathbf{w}_1\|} - \frac{\mathbf{w}_2^\top \mathbf{x} + b_2}{\|\mathbf{w}_2\|} = 0. \quad (5)$$

Then we can define the distance from the sample $\mathbf{x}$ to the two hyperplanes as $D_+(\mathbf{x})$ and $D_-(\mathbf{x})$, respectively. The distance from the sample $\mathbf{x}$ to the two hyperplanes can be expressed as

$$D_+(\mathbf{x}) = \frac{|\frac{\mathbf{w}_1^\top \mathbf{x} + b_1}{\|\mathbf{w}_1\|} + \frac{\mathbf{w}_2^\top \mathbf{x} + b_2}{\|\mathbf{w}_2\|}|}{\|\frac{\mathbf{w}_1}{\|\mathbf{w}_1\|} + \frac{\mathbf{w}_2}{\|\mathbf{w}_2\|}\|} = \frac{|\frac{\mathbf{w}_1^\top \mathbf{x} + b_1}{\|\mathbf{w}_1\|} + \frac{\mathbf{w}_2^\top \mathbf{x} + b_2}{\|\mathbf{w}_2\|}|}{\sqrt{2 + \frac{2(\mathbf{w}_1^\top \mathbf{w}_2)}{\|\mathbf{w}_1\|\|\mathbf{w}_2\|}}}, \quad (6)$$

and

$$D_-(\mathbf{x}) = \frac{|\frac{\mathbf{w}_1^\top \mathbf{x} + b_1}{\|\mathbf{w}_1\|} - \frac{\mathbf{w}_2^\top \mathbf{x} + b_2}{\|\mathbf{w}_2\|}|}{\|\frac{\mathbf{w}_1}{\|\mathbf{w}_1\|} - \frac{\mathbf{w}_2}{\|\mathbf{w}_2\|}\|} = \frac{|\frac{\mathbf{w}_1^\top \mathbf{x} + b_1}{\|\mathbf{w}_1\|} - \frac{\mathbf{w}_2^\top \mathbf{x} + b_2}{\|\mathbf{w}_2\|}|}{\sqrt{2 - \frac{2(\mathbf{w}_1^\top \mathbf{w}_2)}{\|\mathbf{w}_1\|\|\mathbf{w}_2\|}}}. \quad (7)$$

So the two relevant quantities $d(\mathbf{x})$ and $D(\mathbf{x})$ can be defined by

$$d(\mathbf{x}) = \min\{D_+(\mathbf{x}), D_-(\mathbf{x})\}, \quad (8)$$

and

$$D(\mathbf{x}) = \max\{D_+(\mathbf{x}), D_-(\mathbf{x})\}. \quad (9)$$

For a sample $\mathbf{x}$ assigned to class +1, it is easy to conclude that $\mathbf{x}$ is more likely belong to class 1 when either $d(\mathbf{x})$ or $\frac{d(\mathbf{x})}{D(\mathbf{x})}$ becomes larger. So it's natural to associate the probability of the sample $\mathbf{x}$ belonging to positive class with $f(\mathbf{x})$ defined by

$$f(\mathbf{x}) = \begin{cases} d(\mathbf{x})\left(\frac{d(\mathbf{x})}{D(\mathbf{x})}\right)^{\alpha_p} & \frac{|\frac{\mathbf{w}_1^\top \mathbf{x} + b_1}{\|\mathbf{w}_1\|} + \frac{\mathbf{w}_2^\top \mathbf{x} + b_2}{\|\mathbf{w}_2\|}}{\|\frac{\mathbf{w}_1}{\|\mathbf{w}_1\|} + \frac{\mathbf{w}_2}{\|\mathbf{w}_2\|}\|} \cdot \frac{\|\mathbf{w}_2\|}{\|\mathbf{w}_1\|} < 1, \\ 0 & \frac{|\frac{\mathbf{w}_1^\top \mathbf{x} + b_1}{\|\mathbf{w}_1\|} + \frac{\mathbf{w}_2^\top \mathbf{x} + b_2}{\|\mathbf{w}_2\|}}{\|\frac{\mathbf{w}_1}{\|\mathbf{w}_1\|} + \frac{\mathbf{w}_2}{\|\mathbf{w}_2\|}\|} \cdot \frac{\|\mathbf{w}_2\|}{\|\mathbf{w}_1\|} = 1, \\ -d(\mathbf{x})\left(\frac{d(\mathbf{x})}{D(\mathbf{x})}\right)^{\alpha_p} & \frac{|\frac{\mathbf{w}_1^\top \mathbf{x} + b_1}{\|\mathbf{w}_1\|} + \frac{\mathbf{w}_2^\top \mathbf{x} + b_2}{\|\mathbf{w}_2\|}}{\|\frac{\mathbf{w}_1}{\|\mathbf{w}_1\|} + \frac{\mathbf{w}_2}{\|\mathbf{w}_2\|}\|} \cdot \frac{\|\mathbf{w}_2\|}{\|\mathbf{w}_1\|} > 1, \end{cases} \quad (10)$$

where α_p is a weight parameter. The bigger the value $f(\mathbf{x})$ is, the more positive the $\mathbf{x}$ is. And the range of the values of $f(\mathbf{x})$ is $(-\infty, +\infty)$. Instead of directly estimating the posteriori probability in TSVM, we construct

$$P(y = +1|f(\mathbf{x})) = \frac{1}{1 + e^{af(\mathbf{x})+b}}, \quad (11)$$

$$P(y = -1|f(\mathbf{x})) = 1 - P(y = +1|f(\mathbf{x})), \quad (12)$$

where a and b are parameters. Based on the maximum likelihood estimation in [22], the parameters a and b in (11) and (12) can be derived through the application of numerous regularization methods.

2.3. Multi-label twin support vector machine

TSVM [17] was originally proposed for single-label binary classification, constructing two non-parallel hyperplanes for the two classes. To address multi-label problems where each instance may be associated with multiple labels, the multi-label twin support vector machine (MLTSVM) [21] was introduced.

Without loss of generality, assume each instance in the training set in $\mathbb{R}^n$ has K possible labels.

For the k -th label, let $A_k \in \mathbb{R}^{m_{k1} \times n}$ and $B_k \in \mathbb{R}^{m_{k2} \times n}$ denote the data matrices of positive and negative samples, where m_{k1} and m_{k2} are the numbers of positive and negative instances for label k , respectively, and $m = m_{k1} + m_{k2}$. Define $\mathbf{e}_1 \in \mathbb{R}^{m_{k1}}$ and $\mathbf{e}_2 \in \mathbb{R}^{m_{k2}}$ as all-one vectors.

The MLTSVM seeks to construct K proximal hyperplanes, one for each label:

$$f_k(\mathbf{x}) : \mathbf{w}_k^\top \mathbf{x} + b_k = 0, \quad k = 1, \dots, K, \quad (13)$$

where $\mathbf{w}_k$ and b_k are the normal vector and bias term for the k -th label's hyperplane.

For each label k , let $\mathcal{L}_k^+$ and $\mathcal{L}_k^-$ denote the indices of positive and negative samples, respectively.

The matrix-form primal optimization problem for the k -th label is:

$$\begin{aligned} \min_{\mathbf{w}_k, b_k, \xi_{k2}} \quad & \frac{1}{2} \|A_k \mathbf{w}_k + \mathbf{e}_1 b_k\|^2 + c_k \mathbf{e}_2^\top \xi_{k2} + \frac{1}{2} \lambda_k (\|\mathbf{w}_k\|^2 + b_k^2) \\ \text{s.t.} \quad & -(B_k \mathbf{w}_k + \mathbf{e}_2 b_k) + \xi_{k2} \geq \mathbf{e}_2, \quad \xi_{k2} \geq \mathbf{0}, \end{aligned} \quad (14)$$

where $c_k > 0$ is the empirical risk penalty, $\lambda_k > 0$ is the regularization parameter, and $\xi_{k2} \in \mathbb{R}^{m_{k2}}$ are the slack variables for negative samples.

For notational convenience, we define $Q_k = [A_k, \mathbf{e}_1]$ and $P_k = [B_k, \mathbf{e}_2]$, and redefine the parameter vector as $\tilde{\mathbf{w}}_k = [\mathbf{w}_k^\top, b_k]^\top$, so as to eliminate the bias term. Then, the dual optimization problem becomes:

$$\begin{aligned} \max_{\alpha_k} \quad & \mathbf{e}_2^\top \alpha_k - \frac{1}{2} c_k \alpha_k^\top P_k (Q_k^\top Q_k + \lambda_k I)^{-1} P_k^\top \alpha_k \\ \text{s.t.} \quad & \mathbf{0} \leq \alpha_k \leq \mathbf{e}_2, \end{aligned} \quad (15)$$

where $\alpha_k \in \mathbb{R}^{m_{k2}}$ are the Lagrangian multipliers.

Once the dual solution α_k is obtained, the k -th proximal hyperplane can be recovered by:

$$\tilde{\mathbf{w}}_k = -(Q_k^\top Q_k + \lambda_k I)^{-1} P_k^\top \alpha_k. \quad (16)$$

For a new unseen instance $\mathbf{x}_t \in \mathbb{R}^n$, its distance to the k -th hyperplane (13) is given by:

$$d_k(\mathbf{x}_t) = \frac{|\mathbf{w}_k^\top \mathbf{x}_t + b_k|}{\|\mathbf{w}_k\|}, \quad k = 1, \dots, K. \quad (17)$$

If $d_k(\mathbf{x}_t) \leq \Delta_{set}$, where $\Delta_{set} = \min_{k=1,\dots,K} \left(\frac{1}{\|\mathbf{w}_k\|} \right)$, then $\mathbf{x}_t$ is assigned to the k -th label. If there is no $d_k(\mathbf{x}_t) \leq \Delta_{set}$, then $\mathbf{x}_t$ is assigned to the label $\arg \min_{k=1,\dots,K} d_k(\mathbf{x}_t)$.

3. Multi-label Twin support vector machine with probabilistic output

3.1. Linear case

Inspired by MLTSVM and the probabilistic output model of TSVM, we propose a multi-label TSVM with probabilistic output, which follows the classical TSVM formulation. For the k -th label classification problem, the method realizes the classification of multi-labeled data by constructing two non-parallel hyperplanes. The introduction of the regularization term $\frac{1}{2}\lambda_k(\|\mathbf{w}_k\|^2 + b_k^2)$, which was first proposed and justified by Shao et al. [33], has since been widely adopted in TSVM-related algorithms. $\lambda_k > 0$ is the regularization parameter for the k -th label. This regularization not only improves the model's generalization ability, but also ensures the existence of matrix inverses and guarantees the well-posedness of the optimization problem. The specific forms are as follows:

$$\min_{\mathbf{w}_{k1}, b_{k1}, \xi_{k2}} \frac{1}{2} \|A_k \mathbf{w}_{k1} + \mathbf{e}_1 b_{k1}\|^2 + c_{k1} \mathbf{e}_2^\top \xi_{k2} + \frac{1}{2} \lambda_k (\|\mathbf{w}_{k1}\|^2 + b_{k1}^2) \quad (18)$$

$$\text{s.t.} \quad -(B_k \mathbf{w}_{k1} + \mathbf{e}_2 b_{k1}) + \xi_{k2} \geq \mathbf{e}_2, \quad \xi_{k2} \geq 0,$$

and

$$\min_{\mathbf{w}_{k2}, b_{k2}, \xi_{k1}} \frac{1}{2} \|B_k \mathbf{w}_{k2} + \mathbf{e}_2 b_{k2}\|^2 + c_{k2} \mathbf{e}_1^\top \xi_{k1} + \frac{1}{2} \lambda_k (\|\mathbf{w}_{k2}\|^2 + b_{k2}^2) \quad (19)$$

$$\text{s.t.} \quad (A_k \mathbf{w}_{k2} + \mathbf{e}_1 b_{k2}) + \xi_{k1} \geq \mathbf{e}_1, \quad \xi_{k1} \geq 0.$$

The positive class hyperplane $f_{k1}(\mathbf{x}) = \mathbf{w}_{k1}^\top \mathbf{x} + b_{k1}$ and the negative class hyperplane $f_{k2}(\mathbf{x}) = \mathbf{w}_{k2}^\top \mathbf{x} + b_{k2}$ tend to categorize positive and negative class samples to their corresponding sides, respectively.

Here, for each label, we let $Q_k = [A_k, \mathbf{e}_1]$ and $P_k = [B_k, \mathbf{e}_2]$, so as to eliminate the bias terms. Then, the dual optimization problems become:

$$\min_{\alpha_k} \frac{1}{2} c_{k1} \alpha_k^\top P_k (Q_k^\top Q_k + \lambda_k I)^{-1} P_k^\top \alpha_k - \mathbf{e}_2^\top \alpha_k \quad (20)$$

$$\text{s.t.} \quad 0 \leq \alpha_k \leq e_2,$$

and

$$\min_{\gamma_k} \frac{1}{2} c_{k2} \gamma_k^\top Q_k (P_k^\top P_k + \lambda_k I)^{-1} Q_k^\top \gamma_k - \mathbf{e}_1^\top \gamma_k \quad (21)$$

$$\text{s.t.} \quad 0 \leq \gamma_k \leq e_1,$$

The corresponding parametric solutions are:

$$\begin{bmatrix} \mathbf{w}_{k1} \\ b_{k1} \end{bmatrix} = -c_{k1} (Q_k^\top Q_k + \lambda_k I)^{-1} P_k^\top \alpha_k,$$

and

$$\begin{bmatrix} \mathbf{w}_{k2} \\ b_{k2} \end{bmatrix} = c_{k2} (P_k^\top P_k + \lambda_k I)^{-1} Q_k^\top \gamma_k.$$

Here I is the unit matrix, and the introduction of the regularization term λ_k corrects the singularity problem of the matrices $Q_k^\top Q_k$ and $P_k^\top P_k$ to ensure the stability of the solution. The Lagrange multipliers α_k and γ_k correspond to the constraints on the negative and positive class samples, respectively, and their upper and lower bounds are controlled by the penalty parameters c_{k1} and c_{k2} .

After obtaining the parameters of the two hyperplanes, we further propose a probabilistic output mechanism based on a distance metric, analogous to the probabilistic output design for standard TSVM. Specifically, for the k th label, given a sample $\mathbf{x}$, the geometric distances d_1 and d_2 from $\mathbf{x}$ to the positive and negative class hyperplanes, respectively, are computed as:

$$d_1 = \frac{|\mathbf{w}_{k1}^\top \mathbf{x} + b_{k1}|}{\|\mathbf{w}_{k1}\|}, \quad d_2 = \frac{|\mathbf{w}_{k2}^\top \mathbf{x} + b_{k2}|}{\|\mathbf{w}_{k2}\|}. \quad (22)$$

Here, d_1 and d_2 represent the signed Euclidean distances from the sample $\mathbf{x}$ to each of the two hyperplanes, which are used as the basis for probabilistic prediction.

Following the same formulation as in the single-label case, the adjusted bounding distances $D_+^{(k)}(\mathbf{x})$ and $D_-^{(k)}(\mathbf{x})$ are given by:

$$D_{\pm}^{(k)}(\mathbf{x}) = \frac{\left| \frac{\mathbf{w}_{k1}^\top \mathbf{x} + b_{k1}}{\|\mathbf{w}_{k1}\|} \pm \frac{\mathbf{w}_{k2}^\top \mathbf{x} + b_{k2}}{\|\mathbf{w}_{k2}\|} \right|}{\sqrt{2 \pm \frac{2(\mathbf{w}_{k1}^\top \mathbf{w}_{k2})}{\|\mathbf{w}_{k1}\| \|\mathbf{w}_{k2}\|}}} \quad (23)$$

where $D_+^{(k)}(\mathbf{x})$ and $D_-^{(k)}(\mathbf{x})$ correspond to the normalized distances from $\mathbf{x}$ to the two separating hyperplanes, accounting for the angular relationship between the hyperplanes. This unified notation with the symbol ensures consistency in the description of both distances. The definitions are analogous to equations (6) and (7) in Section 2, with an additional subscript k to denote the corresponding label.

Based on the above distances, a confidence score $s_k(\mathbf{x})$ is computed which reflects the degree of confidence that the sample possesses label k . This function takes the value of $(-\infty, +\infty)$.

$$s_k(\mathbf{x}) = \begin{cases} d^{(k)}(\mathbf{x}) \left(\frac{d^{(k)}(\mathbf{x})}{D^{(k)}(\mathbf{x})} \right)^{\alpha_p}, & \text{if } \frac{|\mathbf{w}_{k1}^\top \mathbf{x} + b_{k1}|}{\|\mathbf{w}_{k2}^\top \mathbf{x} + b_{k2}\|} \cdot \frac{\|\mathbf{w}_{k2}\|}{\|\mathbf{w}_{k1}\|} < 1, \\ -d^{(k)}(\mathbf{x}) \left(\frac{d^{(k)}(\mathbf{x})}{D^{(k)}(\mathbf{x})} \right)^{\alpha_p}, & \text{else.} \end{cases} \quad (24)$$

where $d^{(k)}(\mathbf{x}) = \min\{D_+^{(k)}, D_-^{(k)}\}$ denotes the minimum bounding distance, $D^{(k)}(\mathbf{x}) = \max\{D_+^{(k)}, D_-^{(k)}\}$ is the maximum bounding distance, and the moderator $\alpha_p > 0$ controls the strength of the effect of relative distance scaling. This parameter can be found through cross-validation. For simplicity, we follow the setting in [22] and set $\alpha_p = 1$.

To obtain well-calibrated probabilistic outputs, we adopt a method similar to the probabilistic output interpretation used in TSVM. Specifically, this approach is inspired by the work of Platt [35], which proposed a sigmoid fitting strategy for mapping decision values to calibrated posterior probabilities:

$$P(y_k = 1|\mathbf{x}) = \frac{1}{1 + e^{a s_k(\mathbf{x}) + b}}, \quad (25)$$

where $a < 0$ and b are parameters to be estimated. This probabilistic modeling allows not only label prediction but also meaningful ranking among multiple labels. To mitigate the adverse effects of imbalanced class distributions, the target values t_i for sigmoid fitting are smoothed, following Platt's suggestion, as:

$$t_i = \begin{cases} \frac{m_{k1} + 1}{m_{k1} + 2}, & i \in \mathcal{L}_k^+, \\ \frac{1}{m_{k2} + 2}, & i \in \mathcal{L}_k^-, \end{cases} \quad (26)$$

where m_{k1} and m_{k2} denote the number of positive and negative samples for label k , $\mathcal{L}_k^+$ and $\mathcal{L}_k^-$ denote the positive and negative index sets for the k -th label, respectively. The sigmoid parameters (a, b) are obtained by minimizing the negative log-likelihood function:

$$F(a, b) = - \sum_{i=1}^m [t_i \log p_i + (1 - t_i) \log(1 - p_i)], \quad (27)$$

where $p_i = \frac{1}{1 + e^{as_k(\mathbf{x}_i) + b}}$.

The solution to this convex optimization problem has been extensively discussed in the literature and can be solved using Newton's method. At each iteration, the first-order derivative and second-order Hessian are analytically calculated, and the parameters are updated by solving a simple 2×2 linear system. Due to the fact that optimization only involves two scalar variables, the computational cost per iteration is $O(m)$, making it negligible compared to the $O(m^3)$ complexity of dual SVM training. After solving the above optimization problem, all the parameters in (25) are known. We can then compute the probability p that a test instance $\mathbf{x}$ has label y_k according to (25). If $p > 0.5$, $\mathbf{x}$ is considered to have this label. The overall procedure of the proposed Probabilistic Multi-label Twin Support Vector Machine (PMLTSVM) is summarized in Algorithm 1.

In summary, PMLTSVM not only inherits the efficient hyperplane construction of classical TSVM, but also integrates a principled probabilistic output mechanism. This design enables simultaneous multi-label prediction and reliable label ranking, making the model highly flexible and effective for multi-label learning tasks.

3.2. Nonlinear case

To handle nonlinearly separable data, we map samples into a high-dimensional feature space via the kernel trick. For the k -th label, define the kernel-generated surfaces:

$$f_{k1}(\mathbf{x}) = K(\mathbf{x}, C_k^\top) \mathbf{u}_{k1} + b_{k1} = 0, \quad (28)$$

and

$$f_{k2}(\mathbf{x}) = K(\mathbf{x}, C_k^\top) \mathbf{u}_{k2} + b_{k2} = 0, \quad (29)$$

where $C_k = [A_k; B_k]^\top$ is the kernel matrix composed of all samples in the k -th class, and $K : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ is the chosen kernel function. The extended optimization problems for the k -th label become:

algorithm 1 Probabilistic Multi-label Twin Support Vector Machine (PMLTSVM)

Input: Training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$; regularization parameters λ_k , penalty parameters c_{k1}, c_{k2} ; scaling factor α_p .

Output: Predicted labels $\hat{y}_k(\mathbf{x}_t)$ and posterior scores $P(y_k = 1|\mathbf{x}_t)$ for each label $k = 1, \dots, K$.

```

1: for each label  $k = 1$  to  $K$  do
2:   Construct  $A_k, B_k$ .
3:   Solve (20), (21) to obtain  $(\mathbf{w}_{k1}, b_{k1}), (\mathbf{w}_{k2}, b_{k2})$ .
4:   for each training sample  $\mathbf{x}_i$  do
5:     Compute  $d_1^{(k)}(\mathbf{x}_i), d_2^{(k)}(\mathbf{x}_i)$  using (22).
6:     Compute  $D_+^{(k)}(\mathbf{x}_i)$  and  $D_-^{(k)}(\mathbf{x}_i)$  using (23).
7:     Compute the confidence score  $s_k(\mathbf{x}_i)$  using (24).
8:   end for
9:   Fit the sigmoid by minimizing (27) to obtain  $a, b$  via Newton's method.
10:  for each test sample  $\mathbf{x}_t$  do
11:    Compute  $d_1^{(k)}(\mathbf{x}_t), d_2^{(k)}(\mathbf{x}_t)$  using (22).
12:    Compute  $D_+^{(k)}(\mathbf{x}_t)$  and  $D_-^{(k)}(\mathbf{x}_t)$  using (23).
13:    Predict label:  $\hat{y}_k(\mathbf{x}_t) = \mathbb{I}(d_1^{(k)}(\mathbf{x}_t) < d_2^{(k)}(\mathbf{x}_t))$ , with  $\mathbb{I}$  denoting the indicatrix function.
14:    Compute confidence score  $s_k(\mathbf{x}_t)$  using (24).
15:    Compute posterior probability:  $P(y_k = 1|\mathbf{x}_t)$  using (25).
16:  end for
17: end for

```

$$\begin{aligned} \min_{\mathbf{u}_{k1}, b_{k1}, \xi_{k2}} \quad & \frac{1}{2} \|K(A_k, C_k^\top) \mathbf{u}_{k1} + e_1 b_{k1}\|^2 + c_{k1} e_2^\top \xi_{k2} \\ & + \frac{1}{2} \lambda_k (\|\mathbf{u}_{k1}\|^2 + b_{k1}^2) \\ \text{s.t.} \quad & -(K(B_k, C_k^\top) \mathbf{u}_{k1} + e_2 b_{k1}) + \xi_{k2} \geq e_2, \quad \xi_{k2} \geq 0, \end{aligned} \quad (30)$$

and

$$\begin{aligned} \min_{\mathbf{u}_{k2}, b_{k2}, \xi_{k1}} \quad & \frac{1}{2} \|K(B_k, C_k^\top) \mathbf{u}_{k2} + e_2 b_{k2}\|^2 + c_{k2} e_1^\top \xi_{k1} \\ & + \frac{1}{2} \lambda_k (\|\mathbf{u}_{k2}\|^2 + b_{k2}^2) \\ \text{s.t.} \quad & (K(A_k, C_k^\top) \mathbf{u}_{k2} + e_1 b_{k2}) + \xi_{k1} \geq e_1, \quad \xi_{k1} \geq 0. \end{aligned} \quad (31)$$

By introducing Lagrange multipliers α_k and γ_k , we obtain the dual problems:

$$\begin{aligned} \min_{\alpha_k} \quad & \frac{1}{2} c_{k1} \alpha_k^\top P_k (Q_k^\top Q_k + \lambda_k I)^{-1} P_k^\top \alpha_k - e_2^\top \alpha_k \\ \text{s.t.} \quad & 0 \leq \alpha_k \leq e_2, \end{aligned} \quad (32)$$

and

$$\begin{aligned} \min_{\gamma_k} \quad & \frac{1}{2} c_{k2} \gamma_k^\top Q_k (P_k^\top P_k + \lambda_k I)^{-1} Q_k^\top \gamma_k - e_1^\top \gamma_k \\ \text{s.t.} \quad & 0 \leq \gamma_k \leq e_1. \end{aligned} \quad (33)$$

where $P_k = [K(B_k, C_k^\top) e_2]$ and $Q_k = [K(A_k, C_k^\top) e_1]$. Similar to the linear case, in the kernel space, the normalized distances from sample $\mathbf{x}$ to the positive and negative surfaces are:

$$D_{\pm}^{(k)}(\mathbf{x}) = \frac{\left| \frac{\mathbf{u}_{k1}^T K(\mathbf{x}, C_k^T) + b_{k1}}{\sqrt{\mathbf{u}_{k1}^T K(C_k, C_k^T) \mathbf{u}_{k1}}} \pm \frac{\mathbf{u}_{k2}^T K(\mathbf{x}, C_k^T) + b_{k2}}{\sqrt{\mathbf{u}_{k2}^T K(C_k, C_k^T) \mathbf{u}_{k2}}} \right|}{\sqrt{2 \pm \frac{2\mathbf{u}_{k1}^T K(C_k, C_k^T) \mathbf{u}_{k2}}{\mathbf{u}_{k1}^T K(C_k, C_k^T) \mathbf{u}_{k1} \cdot \mathbf{u}_{k2}^T K(C_k, C_k^T) \mathbf{u}_{k2}}}}. \quad (34)$$

The posterior probability transformation and parameter optimization follow the same approach as the linear case, maintaining the Sigmoid calibration process from Equations (24)-(27).

4. Safe screening rules for PMLTSVM

4.1. Dual property analysis

To further explore the solution structure and sparsity properties of the proposed PMLTSVM model, we begin with the Karush–Kuhn–Tucker (KKT) conditions of the dual optimization problems. By examining the behavior of each training instance in the dual space, we establish the theoretical foundation for the development of the Safe Screening Rule (SSR).

For clarity, we focus on the optimization problem associated with the positive-class hyperplane. Since the SSR formulation can be independently applied to each label, it suffices to present the derivation for an arbitrary label, say the k -th one. To simplify the notation and avoid unnecessary subscripts, we denote by m_1 and m_2 the numbers of positive and negative samples, respectively, for the current label under consideration. The Lagrangian of the dual problem (33) is given by

$$L_{k1} = \frac{1}{2} c_{k1} \alpha_k^T P_k (Q_k^T Q_k + \lambda_k I)^{-1} P_k^T \alpha_k - e_2^T \alpha_k - k_1^T (\alpha_k - 0) - k_2^T (e_2 - \alpha_k), \quad (35)$$

where $k_1, k_2 \geq 0$ are the Lagrangian multipliers. Differentiating with respect to α_k and applying the complementary slackness conditions yields the KKT system:

$$\frac{\partial L_{k1}}{\partial \alpha_k} = c_{k1} P_k (Q_k^T Q_k + \lambda_k I)^{-1} P_k^T \alpha_k^* - e_2 - k_1^* + k_2^*, \quad (36)$$

$$(k_1^*)_i ((\alpha_k^*)_i - 0) = 0, \quad i = 1, \dots, m_2, \quad (37)$$

$$(k_2^*)_i (1 - (\alpha_k^*)_i) = 0, \quad i = 1, \dots, m_2, \quad (38)$$

$$0 \leq \alpha_k^* \leq e_2. \quad (39)$$

Here $(k_1^*)_i$ denotes the i -th entry of the vector k_1^* , and analogously for other terms. Define

$$F^1 = c_{k1} P_k (Q_k^T Q_k + \lambda_k I)^{-1} P_k^T \alpha_k^*. \quad (40)$$

From (36), if the i -th component satisfies $F_i^1 > 1$ ($i = 1, \dots, m_2$), then $(k_1^*)_i > (k_2^*)_i$, and by (37) it follows that $(\alpha_k^*)_i = 0$. A similar argument applies to the cases $F_i^1 = 1$ and $F_i^1 < 1$, leading to the following characterization:

$$\begin{cases} F_i^1 > 1 & \implies (k_1^*)_i > (k_2^*)_i & \implies (\alpha_k^*)_i = 0, \\ F_i^1 = 1 & \implies (k_1^*)_i = (k_2^*)_i & \implies 0 \leq (\alpha_k^*)_i \leq 1, \\ F_i^1 < 1 & \implies (k_1^*)_i < (k_2^*)_i & \implies (\alpha_k^*)_i = 1. \end{cases} \quad (41)$$

Therefore, identifying the zero and one components of α_k^* reduces to determining the relative position of F_i^1 with respect to 1. Similarly, we can derive the corresponding equations for γ_k^* .

4.2. Screening strategy for parameters c_{k1} and c_{k2}

In the preceding discussion, we have established that solving TSVM essentially reduces to addressing the quadratic programming problems (20) and (21). Taking problem (20) as an example, the computational complexity is largely determined by the dimension of the vector α_k . The key idea of SSR is to exploit condition (41) to pre-identify the components of the optimal solution α_k^* that will take values 0 or 1, thereby significantly reducing the size of the optimization problem and accelerating the solution process.

However, directly applying (41) for this purpose is infeasible. The reason is that it involves the quantity $F^1 = c_{k1} P_k (Q_k^T Q_k + \lambda_k I)^{-1} P_k^T \alpha_k^*$, which itself depends on the unknown α_k^* , and α_k^* can only be obtained by fully solving the original quadratic programming problem. To overcome this difficulty, SSR constructs a region Θ that is guaranteed to contain the optimal solution α_k^* . By estimating the upper and lower bounds of relevant expressions within this region, safe screening decisions can be made without explicitly knowing the optimal solution, thereby ensuring both correctness and computational efficiency.

Since the SSRs corresponding to each label can be processed in parallel, we omit the subscript k in c_k , α_k , and related terms for simplicity in the following derivation of the SSRs. To construct such a region Θ , we introduce the following lemmas:

Lemma 1 (Variational Inequality). Let $P \subset \mathbb{R}^n$ be a convex set, and suppose that $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable on an open set containing P . If x^* is a local minimizer of f on P , then

$$\langle \nabla f(x^*), x - x^* \rangle \geq 0, \quad \forall x \in P. \quad (42)$$

Based on the above lemma, one can estimate the region Θ containing the optimal solution $\alpha^*(c_b)$ by using a known solution $\alpha^*(c_a)$ corresponding to a smaller parameter $c_a < c_b$, assuming all other parameters are fixed. This leads to the following theorem.

Theorem 1. Let the feasible regions of problems (20) and (21) be convex. Given $c_b > c_a > 0$ and their corresponding optimal solutions $\alpha^*(c_a)$ and $\gamma^*(c_a)$, the following inequalities hold:

$$\begin{aligned} \left\| J_1^T \alpha^*(c_b) - \frac{c_b + c_a}{2c_b} J_1^T \alpha^*(c_a) \right\| &\leq \left\| \frac{c_b - c_a}{2c_b} J_1^T \alpha^*(c_a) \right\|, \\ \left\| J_2^T \gamma^*(c_b) - \frac{c_b + c_a}{2c_b} J_2^T \gamma^*(c_a) \right\| &\leq \left\| \frac{c_b - c_a}{2c_b} J_2^T \gamma^*(c_a) \right\|. \end{aligned}$$

where

$$H = P(Q^T Q + \lambda_k I)^{-1} P^T, \quad R = Q(P^T P + \lambda_k I)^{-1} Q^T \quad (43)$$

and $H = J_1 J_1^T$, $R = J_2 J_2^T$, and J_1 and J_2 are matrices obtained via Cholesky decomposition.

Proof. We only provide the derivation for the first inequality, as the second follows analogously. Considering problem (20) with known c_a and corresponding solution $\alpha^*(c_a)$, we define the objective function $f(\alpha; c_1) = \frac{1}{2} \alpha^T c_1 H \alpha - \alpha^T e$, whose gradient is given by $\nabla f(\alpha) = c_1 H \alpha - e$. Since H is positive definite, f is convex and its optimal solution satisfies the first-order condition.

By Lemma 1 (Variational Inequality), we have:

$$\begin{aligned}\langle \nabla f(\alpha^*(c_b)), \alpha^*(c_a) - \alpha^*(c_b) \rangle &\geq 0, \\ \langle \nabla f(\alpha^*(c_a)), \alpha^*(c_b) - \alpha^*(c_a) \rangle &\geq 0.\end{aligned}$$

Substituting the gradient:

$$\begin{aligned}\langle c_b H \alpha^*(c_b) - c_a H \alpha^*(c_a), \alpha^*(c_a) - \alpha^*(c_b) \rangle &\geq 0, \\ \implies c_b (\alpha^*(c_b))^T H \alpha^*(c_b) + c_a (\alpha^*(c_a))^T H \alpha^*(c_a) \\ &\leq c_b (\alpha^*(c_a))^T H \alpha^*(c_b) + c_a (\alpha^*(c_b))^T H \alpha^*(c_a).\end{aligned}$$

Rewriting this:

$$\begin{aligned}(\alpha^*(c_b))^T H \alpha^*(c_b) - \frac{c_b + c_a}{c_b} (\alpha^*(c_a))^T H \alpha^*(c_b) \\ \leq -\frac{c_a}{c_b} (\alpha^*(c_a))^T H \alpha^*(c_a).\end{aligned}$$

Substituting $H = J_1 J_1^T$, we observe that each quadratic term can be expressed in terms of the Euclidean norm, e.g., $(\alpha^*(c_b))^T H \alpha^*(c_b) = \|J_1^T \alpha^*(c_b)\|^2$. Thus, the above inequality becomes:

$$\|J_1^T \alpha^*(c_b)\|^2 - \frac{c_b + c_a}{c_b} \langle J_1^T \alpha^*(c_a), J_1^T \alpha^*(c_b) \rangle + \frac{c_a}{c_b} \|J_1^T \alpha^*(c_a)\|^2 \leq 0.$$

Completing the square with respect to $J_1^T \alpha^*(c_b)$ yields:

$$\left\| J_1^T \alpha^*(c_b) - \frac{c_b + c_a}{2c_b} J_1^T \alpha^*(c_a) \right\|^2 \leq \left(\frac{c_b - c_a}{2c_b} \right)^2 \|J_1^T \alpha^*(c_a)\|^2.$$

Therefore, we arrive at the following bound:

$$\left\| J_1^T \alpha^*(c_b) - \frac{c_b + c_a}{2c_b} J_1^T \alpha^*(c_a) \right\| \leq \left\| \frac{c_b - c_a}{2c_b} J_1^T \alpha^*(c_a) \right\| \quad (44)$$

This inequality has a clear geometric interpretation: it states that the unknown vector $J_1^T \alpha^*(c_b)$ must lie inside a closed Euclidean ball whose center and radius can be computed directly from the known solution $J_1^T \alpha^*(c_a)$. This ball constraint is the key to deriving the subsequent safe screening rules.

More precisely, $J_1^T \alpha^*(c_b)$ lies within a closed ball $\Theta(o, r)$ centered at

$$o := \frac{c_b + c_a}{2c_b} J_1^T \alpha^*(c_a), \quad r := \frac{c_b - c_a}{2c_b} \|J_1^T \alpha^*(c_a)\|.$$

To utilize this bound for screening, we parameterize any possible point in the ball as

$$\theta = o + v, \quad \text{with} \quad \|v\| \leq r.$$

This representation separates the known part o from the uncertain deviation v whose magnitude is bounded. We now propagate this uncertainty through the gradient expression to bound each component individually.

Consider the i -th component of the gradient of the objective function:

$$F_i^1 = c_b [H \alpha^*(c_b)]_i = c_b [J_1 J_1^T \alpha^*(c_b)]_i = c_b [J_1]_i (o + v).$$

Here $[J_1]_i$ denotes the i -th row of J_1 , so $[J_1]_i (o + v)$ is simply the dot product between this row and the vector $(o + v)$. Since v is

bounded in norm, we can apply the Cauchy-Schwarz inequality to bound the extreme values of F_i^1 .

By the Cauchy-Schwarz inequality, we get:

$$\begin{aligned}\inf F_i^1 &\geq \frac{c_b + c_a}{2} [H \alpha^*(c_a)]_i - \frac{c_b - c_a}{2} \| [J_1]_i \| \cdot \| J_1^T \alpha^*(c_a) \|, \\ \sup F_i^1 &\leq \frac{c_b + c_a}{2} [H \alpha^*(c_a)]_i + \frac{c_b - c_a}{2} \| [J_1]_i \| \cdot \| J_1^T \alpha^*(c_a) \|.\end{aligned} \quad (45)$$

These bounds give us a guaranteed interval containing F_i^1 without ever computing $\alpha^*(c_b)$ explicitly. If the entire interval lies strictly above or below a threshold, we can safely determine the activity of the corresponding dual variable in advance.

These bounds enable safe screening decisions based on the known solution $\alpha^*(c_a)$.

Corollary 1 (SSR- $c_1(c_2)$) Suppose a sequence of parameters is given as:

$$\begin{aligned}0 < c_1^1 < c_1^2 < \dots < c_1^k < c_1^{k+1} < \dots < c_1^d, \\ 0 < c_2^1 < c_2^2 < \dots < c_2^k < c_2^{k+1} < \dots < c_2^d.\end{aligned}$$

If the optimal solution $\alpha^*(c_1^k)$ (or $\gamma^*(c_2^k)$) is known for $1 \leq k < d$, we can directly apply the above bounding technique with $c_a = c_1^k$ and $c_b = c_1^{k+1}$ to obtain screening conditions for $\alpha^*(c_1^{k+1})$. The same logic applies for $\gamma^*(c_2^{k+1})$ by symmetry.

Then using eq (41), for $i = 1, \dots, m_2$, the following holds:

$$\begin{aligned}\text{if } \frac{c_1^{k+1} + c_1^k}{2} [H \alpha^*(c_1^k)]_i - \| [J_1]_i \| \cdot \left\| \frac{c_1^{k+1} - c_1^k}{2} J_1^T \alpha^*(c_1^k) \right\| \\ > 1 \implies \alpha_i^*(c_1^{k+1}) = 0,\end{aligned} \quad (46)$$

$$\begin{aligned}\text{if } \frac{c_1^{k+1} + c_1^k}{2} [H \alpha^*(c_1^k)]_i + \| [J_1]_i \| \cdot \left\| \frac{c_1^{k+1} - c_1^k}{2} J_1^T \alpha^*(c_1^k) \right\| \\ < 1 \implies \alpha_i^*(c_1^{k+1}) = 1.\end{aligned} \quad (47)$$

Similarly, for problem (21) and $i = 1, \dots, m_1$:

$$\begin{aligned}\text{if } \frac{c_2^{k+1} + c_2^k}{2} [R \gamma^*(c_2^k)]_i - \| [J_2]_i \| \cdot \left\| \frac{c_2^{k+1} - c_2^k}{2} J_2^T \gamma^*(c_2^k) \right\| \\ > 1 \implies \gamma_i^*(c_2^{k+1}) = 0,\end{aligned} \quad (48)$$

$$\begin{aligned}\text{if } \frac{c_2^{k+1} + c_2^k}{2} [R \gamma^*(c_2^k)]_i + \| [J_2]_i \| \cdot \left\| \frac{c_2^{k+1} - c_2^k}{2} J_2^T \gamma^*(c_2^k) \right\| \\ < 1 \implies \gamma_i^*(c_2^{k+1}) = 1.\end{aligned} \quad (49)$$

This corollary provides a crucial tool for sequential parameter selection. In practice, one often needs to conduct grid search over multiple (c_1, c_2) combinations to select optimal hyperparameters. By applying this corollary, one can directly infer the optimal values of certain variables at c_1^{k+1} (or c_2^{k+1}) based on solutions at c_1^k (or c_2^k), thereby skipping unnecessary computations and significantly reducing the number and cost of optimization problems.

4.3. Screening Strategy for Parameters λ_k

In this section, we present a safe screening rule with respect to the regularization parameter λ_k . The parameter λ_k is closely related to the matrices (43) which appear in the dual problems.

Since the Subproblem of (SSR) corresponding to each label-specific parameter λ_k can be solved in parallel, we omit the label index in λ for notational simplicity. In the following, λ^k denotes the k -th element in the parameter sequence.

Theorem 2. For the dual problems (20) and (21), suppose that the optimal solutions $\alpha^*(\lambda_a)$ and $\gamma^*(\lambda_a)$ are known. Then for any $\lambda_b > \lambda_a > 0$, the following inequalities hold:

$$\begin{aligned} & \left\| S_b^\top \alpha^*(\lambda_b) - S_b^{-1} \cdot \frac{H(\lambda_a) + H(\lambda_b)}{2} \alpha^*(\lambda_a) \right\| \\ & \leq \left\| S_b^{-1} \cdot \frac{H(\lambda_a) - H(\lambda_b)}{2} \alpha^*(\lambda_a) \right\|, \\ & \left\| Z_b^\top \gamma^*(\lambda_b) - Z_b^{-1} \cdot \frac{R(\lambda_a) + R(\lambda_b)}{2} \gamma^*(\lambda_a) \right\| \\ & \leq \left\| Z_b^{-1} \cdot \frac{R(\lambda_a) - R(\lambda_b)}{2} \gamma^*(\lambda_a) \right\|, \end{aligned}$$

where $H(\lambda_b) = S_b S_b^\top$ and $R(\lambda_b) = Z_b Z_b^\top$. The matrices S_b and Z_b can be computed via Cholesky decomposition.

Proof. We only prove the inequality involving $\alpha^*(\lambda)$; the second inequality can be shown similarly. From the variational inequalities associated with the dual problems, we obtain :

$$\langle H(\lambda_a) \alpha^*(\lambda_a) - e_2, \alpha^*(\lambda_b) - \alpha^*(\lambda_a) \rangle \geq 0, \quad (50)$$

$$\langle H(\lambda_b) \alpha^*(\lambda_b) - e_2, \alpha^*(\lambda_a) - \alpha^*(\lambda_b) \rangle \geq 0. \quad (51)$$

Adding the two inequalities yields:

$$\begin{aligned} & (\alpha^*(\lambda_b))^\top H(\lambda_b) \alpha^*(\lambda_b) - (\alpha^*(\lambda_b))^\top (H(\lambda_a) + H(\lambda_b)) \alpha^*(\lambda_a) \\ & + (\alpha^*(\lambda_a))^\top H(\lambda_a) \alpha^*(\lambda_a) \leq 0. \end{aligned}$$

Rearranging terms gives:

$$\begin{aligned} & (\alpha^*(\lambda_b))^\top H(\lambda_b) \alpha^*(\lambda_b) - 2(\alpha^*(\lambda_b))^\top \cdot \frac{H(\lambda_a) + H(\lambda_b)}{2} \alpha^*(\lambda_a) \\ & + (\alpha^*(\lambda_a))^\top H(\lambda_a) \alpha^*(\lambda_a) \leq 0. \end{aligned} \quad (52)$$

This inequality can be transformed into a norm bound by completing the square, which follows a similar idea to the derivation in Section 4.2. Specifically, letting $H(\lambda_b) = S_b S_b^\top$ and applying the change of variables $u_b = S_b^\top \alpha^*(\lambda_b)$, the left-hand side becomes $\|u_b - c\|^2 - r^2 \leq 0$, where c and r denote the center and radius defined below. This step uses the positive definiteness of $H(\lambda_b)$ to ensure the Cholesky factorization exists and the transformation is valid.

Letting $H(\lambda_b) = S_b S_b^\top$, and performing a change of variables via Cholesky decomposition, we complete the square and derive the following bound:

$$\begin{aligned} & \left\| S_b^\top \alpha^*(\lambda_b) - S_b^{-1} \cdot \frac{H(\lambda_a) + H(\lambda_b)}{2} \alpha^*(\lambda_a) \right\| \\ & \leq \left\| S_b^{-1} \cdot \frac{H(\lambda_a) - H(\lambda_b)}{2} \alpha^*(\lambda_a) \right\| \end{aligned}$$

which is equivalent to stating that $S_b^\top \alpha^*(\lambda_b)$ lies in a closed ball centered at $S_b^{-1} \frac{H(\lambda_a) + H(\lambda_b)}{2} \alpha^*(\lambda_a)$ with radius $\left\| S_b^{-1} \frac{H(\lambda_a) - H(\lambda_b)}{2} \alpha^*(\lambda_a) \right\|$. This closed-ball representation provides a clear geometric description of the feasible set, which will be exploited for safe screening. ■

Corollary 2. (SSR- λ_k) By projecting the above closed ball onto each coordinate axis, we can derive explicit upper and lower bounds for each component of $\alpha^*(\lambda_b)$ and $\gamma^*(\lambda_b)$. This projection, together with the Cauchy-Schwarz inequality, allows us

to check whether a variable can be safely fixed to zero or one without solving the optimization problem at λ_b . Consider an increasing sequence of regularization parameters: $0 < \lambda^1 < \lambda^2 < \dots < \lambda^d$. Assume that the optimal dual variables $\alpha^*(\lambda^k)$ and $\gamma^*(\lambda^k)$ for problems (20) and (21), respectively, are known at some index k with $1 \leq k < d$. Then, for each $i \in \{1, 2, \dots, m_2\}$, the following safe screening rules apply:

$$\text{If } \lambda_k \left[\bar{H}^{(k)} \alpha^*(\lambda^k) \right]_i - \lambda_k \| [S_{k+1}]_i \| \cdot \| S_{k+1}^{-1} \Delta H^{(k)} \alpha^*(\lambda^k) \|^2 > 1, \\ \text{then } \alpha_i^*(\lambda^{k+1}) = 0.$$

$$\text{If } \lambda_k \left[\bar{H}^{(k)} \alpha^*(\lambda^k) \right]_i + \lambda_k \| [S_{k+1}]_i \| \cdot \| S_{k+1}^{-1} \Delta H^{(k)} \alpha^*(\lambda^k) \|^2 < 1, \\ \text{then } \alpha_i^*(\lambda^{k+1}) = 1.$$

Similarly, for the dual variable $\gamma^*(\lambda)$, the screening rules are:

$$\text{If } \lambda_k \left[\bar{R}^{(k)} \gamma^*(\lambda^k) \right]_i - \lambda_k \| [Z_{k+1}]_i \| \cdot \| Z_{k+1}^{-1} \Delta R^{(k)} \gamma^*(\lambda^k) \|^2 > 1, \\ \text{then } \gamma_i^*(\lambda^{k+1}) = 0.$$

$$\text{If } \lambda_k \left[\bar{R}^{(k)} \gamma^*(\lambda^k) \right]_i + \lambda_k \| [Z_{k+1}]_i \| \cdot \| Z_{k+1}^{-1} \Delta R^{(k)} \gamma^*(\lambda^k) \|^2 < 1, \\ \text{then } \gamma_i^*(\lambda^{k+1}) = 1.$$

where the midpoint and difference matrices are:

$$\begin{aligned} \bar{H}^{(k)} &= \frac{H(\lambda^k) + H(\lambda^{k+1})}{2}, \quad \Delta H^{(k)} = \frac{H(\lambda^k) - H(\lambda^{k+1})}{2}, \\ \bar{R}^{(k)} &= \frac{R(\lambda^k) + R(\lambda^{k+1})}{2}, \quad \Delta R^{(k)} = \frac{R(\lambda^k) - R(\lambda^{k+1})}{2}. \end{aligned}$$

We can leverage this corollary to accelerate the optimization process by safely screening the dual variables associated with each label-specific regularization parameter λ_k .

4.4. Reduced PMLTSVM

After applying the Safe Screening Rules (SSR) to PMLTSVM, most of the zero and one components of the solutions are safely discarded. Consequently, the TSVM optimization reduces to two simplified quadratic programming problems (QPPs). Let D_p (D_n) denote the indices of positive (negative) samples that have been safely discarded, (the indices corresponding to the 0 and 1 components screened out from α or γ). Similarly, let R_p (R_n) represent the indices of the remaining active positive (negative) samples. Define $H = J_1 J_1^\top$ and $R = J_2 J_2^\top$, where $J_1 = P(Q^\top Q + \lambda_k I)^{-1} P^\top$ and $J_2 = Q(P^\top P + \lambda_k I)^{-1} Q^\top$. Then, the reduced dual problems can be formulated as follows:

$$\max_{\alpha} - \frac{c_1}{2} \alpha_{R_n}^\top H_{R_n \times R_n} \alpha_{R_n} - c_1 \alpha_{D_n}^\top H_{D_n \times R_n} \alpha_{R_n} + (e_2)_{R_n}^\top \alpha_{R_n} \quad (53)$$

$$\text{s.t. } 0 \leq \alpha_{R_n} \leq (e_2)_{R_n},$$

and

$$\max_{\beta} - \frac{c_2}{2} \beta_{R_p}^\top R_{R_p \times R_p} \beta_{R_p} - c_2 \beta_{D_p}^\top R_{D_p \times R_p} \beta_{R_p} + (e_1)_{R_p}^\top \beta_{R_p} \quad (54)$$

$$\text{s.t. } 0 \leq \beta_{R_p} \leq (e_1)_{R_p}.$$

Since the screening process only requires the dual solution at a given parameter value and the parameter itself, the proposed SSR strategy can be integrated with any optimizer. In

practical applications, it is often necessary to select and fine-tune a sequence of continuous parameter values. Let the optimizer be denoted by `solver`. After obtaining the solution at a smaller parameter value, one can utilize both the solution and the parameter to perform safe screening for the next optimization problem, thereby accelerating the solving process. Notably, the computational cost of SSR is significantly lower than that of solving the corresponding quadratic programming problem. The overall procedure is summarized in the algorithm 2 below. The situation of $\text{SSR-}\lambda_k$ is completely consistent, just replace the corresponding SSR method.

algorithm 2 Accelerated Parameter Selection with SSR- c

Input: Sequence of parameters $0 < c^1 < c^2 < \dots < c^d$, fixed matrix H, R defined in Eq (43)

Output: Optimal model and corresponding parameter

```

1: function SOLVER( $H, R, [D_p, D_n], [R_p, R_n], c$ )
2:   #  $c$  is the penalty parameter;  $[D_p, D_n]$  and  $[R_p, R_n]$  are
   Boolean index arrays defined in Eq. (53) and Eq. (54)
3:   return QP solution of problem defined by Eq. (53) and (54)
4: end function
5: function SSR_1( $c_k, c_{k+1}, \alpha_k, H$ )
6:   Compute index set  $I_{0,1}$  of  $\alpha_{k+1}$  by Eq. (46) and (47)
7:   Construct  $D_n, R_n$  based on index set
8:   return  $D_n, R_n$ 
9: end function
10: function SSR_2( $c_k, c_{k+1}, \gamma_k, R$ )
11:   Compute index set  $I_{0,1}$  of  $\gamma_{k+1}$  by Eq. (48) and (49)
12:   Construct  $D_p, R_p$  based on index set
13:   return  $D_p, R_p$ 
14: end function
15: for each label do
16:    $\alpha_1, \gamma_1 \leftarrow \text{SOLVER}(H, R, \text{all}, \text{all}, c^1)$ 
17:   for  $i = 1$  to  $d - 1$  do
18:      $D_n, R_n \leftarrow \text{SSR}_1(c^i, c^{i+1}, \alpha_i, H, R)$ 
19:      $D_p, R_p \leftarrow \text{SSR}_2(c^i, c^{i+1}, \gamma_i, H, R)$ 
20:      $\alpha_{i+1}, \gamma_{i+1} \leftarrow \text{SOLVER}(H, R, [D_p, D_n], [R_p, R_n], c^{i+1})$ 
21:     Evaluate model performance under  $c^{i+1}$ 
22:     if performance at  $c^{i+1}$  is better than  $c^i$  then
23:       Mark  $c^{i+1}$  as the best parameter
24:       Save current model
25:     else
26:       continue
27:     end if
28:   end for
29: end for
30: return Best model and corresponding parameter

```

5. Computational complexity

First, we analyze the computational time complexity of PMLTSVM. As is widely known, the time complexity of TSVM is $O(m^3/4)$. Although PMLTSVM requires solving two smaller QPPs for each label and performing $2L$ matrix inver-

sions (where L denotes the total number of labels), its overall computational complexity remains unchanged. Thus, the time complexity of PMLTSVM is $O(Lm^3/4)$.

Next, we examine the complexity of SSR+PMLTSVM. The SSR can identify inactive samples prior to the solution process, thereby reducing the scale of the dual problem. Moreover, the time consumed in the screening process is negligible. Assuming that 60% of the samples can be screened out, the time complexity of SSR+PMLTSVM is $O(L/4) \times (2/5 \times m)^3 = 2Lm^3/125$, with an acceleration ratio of approximately 15.63 times. In fact, the more samples that are identified and screened out, the greater the acceleration effect. Subsequent experiments will demonstrate that the screening ratio can reach as high as 90%.

6. Numerical experiment

In this section, we first describe the characteristics of the datasets used and the data pre-processing methods. Then, we introduce the evaluation metrics used in the experiments, which can be divided into two categories: bipartitions based measures and ranking based measures. In Section 5.3, we briefly introduce the comparison algorithms used in the experiments, and in Section 5.4, we present the parameter comparison strategies for each algorithm. Finally, we show the experimental results. First, we compare the effectiveness of the algorithms on the benchmark datasets. Then, we demonstrate the effect of the safe screening rule (SSR). Experiments on nine relatively small datasets are conducted on a PC running Windows 11, equipped with an Intel Core i7-12700H CPU (4.70 GHz) and 32.0 GB of RAM, using Python code implemented with the numpy library. For the two large-scale datasets, *delicious* and *corel16k1*, experiments are carried out on a server running Ubuntu 22.04, equipped with a 128-core AMD EPYC 9654 CPU and 240.0 GB of RAM. Code is available at <https://github.com/brandinzhang/PMLTSVM>.

6.1. The description of the datasets and evaluation metrics

In this paper, a total of nine multi label benchmark datasets were used for comparative experiments, and the statistical features of the datasets are presented in Table 1.

We introduce the evaluation metrics used in comparative experiments. In multi label learning, evaluation metrics can be divided into two categories based on the different inputs received. One is to accept the predicted label results (which only contain 0 or 1), such as Hamming loss, F-score, etc; One is to accept the predicted label score or probability (the larger the value, the more likely the instance is to have the label), such as one error, ranking loss, average precision, etc

(i) Hamming Loss

$$hloss(g) = \frac{1}{p} \sum_{i=1}^p \frac{1}{Q} |g(x_i) \triangle Y_i|,$$

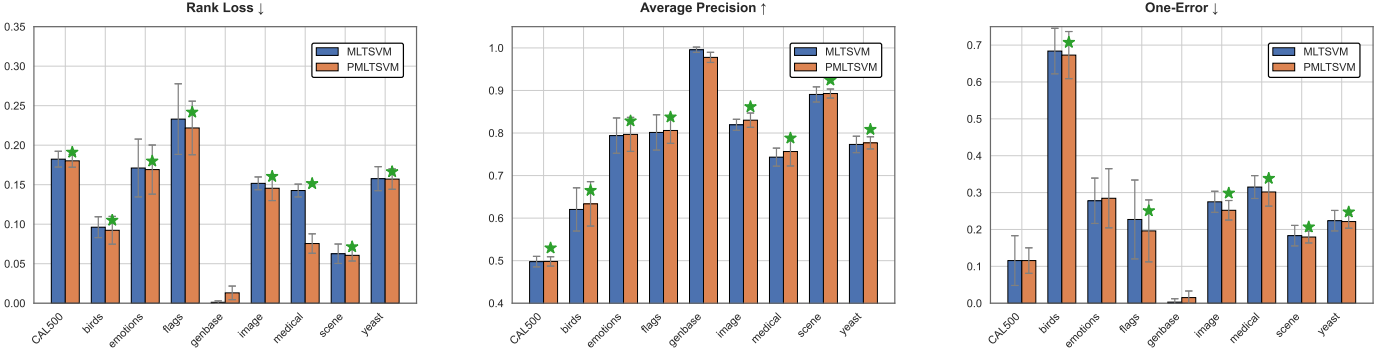


Fig. 1. Performance comparison of MLTSVM and PMLTSVM in three evaluation metrics for measuring label sorting ability. The green asterisk (*) indicates a improvement.

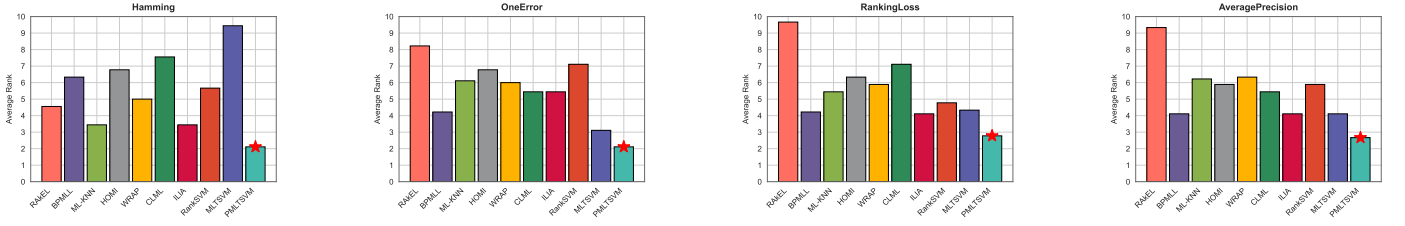


Fig. 2. The average ranking of different models on different evaluation indicators, with a red asterisk (*) indicating that the model has the best average ranking.

Table 1

Statistics for the real-world multi-label datasets used in our experiments.

Dataset	Domain	Instances	Features	Labels	Cardinality	Density
emotions ^a	music	593	72	6	1.869	0.311
birds ^a	audio	645	260	19	1.014	0.053
flags ^a	images	194	19	7	3.392	0.485
yeast ^a	biology	2417	103	14	4.237	0.303
CAL500 ^a	music	502	68	174	26.056	0.149
genbase ^a	biology	662	1186	27	1.920	0.002
medical ^a	medical	978	1449	45	1.224	0.001
scene ^a	images	2407	294	6	1.074	0.179
image ^b	images	2000	294	5	1.226	0.245
delicious ^a	text	16105	500	983	19.020	0.019
corel16k ^a	images	13766	500	153	2.859	0.019

^a Datasets are available at <https://mulan.sourceforge.net/datasets-mlc.html>.

^b Datasets are available at <https://palm.seu.edu.cn/zhangml/>.

where p represents the total number of samples, Q represents the total number of labels, and Δ represents the symmetric difference between the predicted label set $g(\mathbf{x}_i)$ set and the ground-truth label set Y_i . When $|Y| = 1$, the Hamming Loss will be $\frac{2}{Q}$ times of usual misclassification rate. The hamming loss denotes the score of the misclassified sample-label pairs, i.e., an irrelevant label is predicted or a relevant label is not predicted. Note that the value of hamming loss is proportional to the similarity between $g(\mathbf{x}_i)$ and Y_i .

(ii) One-error

$$one - error(f) = \frac{1}{p} \sum_{i=1}^p \frac{1}{|Y_i|} \sum_{y \in Y_i} \mathbb{I}[\arg \max_{y \in \mathcal{Y}} f(\mathbf{x}_i, y) \neq y],$$

where $\mathcal{Y}$ represents the set of all labels, and the function f is defined by $f(\mathbf{x}_i, y) = \mathbf{w}_y^\top \mathbf{x}_i + b_y$. The one-error measures the fraction of examples whose top-ranked label is not in the truth label set. Obviously, the performance of the model can be reflected by the value of one-error. Note that, when the number of labels is one, one-error is a traditional classification error.

(iii) Ranking Loss

$$rloss(f) = \frac{1}{p} \sum_{i=1}^p \frac{1}{|Y_i|} \sum_{y' \in \bar{Y}_i} \mathbb{I}[f(\mathbf{x}_i, y') \leq f(\mathbf{x}_i, y'')],$$

where $\bar{Y}_i$ is the complementary set of Y_i in $\mathcal{Y}$, y' and y'' belong to $\bar{Y}_i$, Y_i , respectively. The ranking loss indicates the number of times that relevant labels are ranked lower than irrelevant labels. The performance is better when the value of $rloss(f)$ is smaller. Until $rloss(f) = 0$, the performance will be perfect.

(iv) Average Precision

$$avp(f) = \frac{1}{p} \sum_{i=1}^p \frac{1}{|Y_i|} \sum_{y \in Y_i} \frac{\mathbb{I}[rank_f(\mathbf{x}_i, y) \leq r]}{r},$$

where $r = rank_f(\mathbf{x}_i, y)$. This evaluation metric evaluates the average fraction of relevant labels ranked higher than a particular label $y \in Y_i$. The performance is better when the $avp(f)$ is bigger. Until $avp(f) = 1$, the performance will be perfect.

6.2. Comparison algorithms used in the experiments

- **ML-KNN**[10] predicts labels by statistically analyzing the distribution probability of labels in the neighborhood of target samples.
- **BPMLL**[36] optimizes multi-label ranking tasks through modified neural network architectures. It adjusts connection weights via error backpropagation to handle complex non-linear relationships between labels.
- **RankSVM**[13] extends Support Vector Machines to multi-label problems by reformulating them as label ranking tasks. It maximizes ranking margins between relevant and irrelevant labels through structural risk minimization.
- **RAKEL**[9] (Random k-Label Sets) constructs multiple binary classifiers through random selection of label subsets. It alleviates the curse of dimensionality in label space by ensemble learning strategies.
- **MLTSVM**[21] enhances classification boundaries by constructing twin non-parallel hyperplanes for each label.
- **HOMI**[38] captures high-order label correlations through implicit relationship modeling. It integrates matrix factorization techniques with high-rank label interaction analysis to address complex dependency patterns.
- **WRAP** [40] generates label-specific features in an embedded feature space to improve classification performance in multi-label scenarios.
- **CLML** [41] learns both common and label-specific features by exploiting correlation information between labels and instances.
- **ILIA** [42] (BRKNN-ILIA) leverages implicit Relative Labeling Importance (RLI) in multi-label metric learning. It recovers RLIs by transferring manifold structures from the feature space to the label space, and aligns predictive outputs with recovered RLIs to enhance similarity metrics between multi-label instances.

6.3. Parameter selection

The parameter selection process is rigorously conducted through 10-fold cross-validation to ensure model robustness and generalizability. For MLTSVM and PMLTSVM, we unify the penalty parameters by setting $c_k = c$ and $\lambda_k = \lambda$ across all labels to maintain algorithmic consistency. In ML-KNN, the neighborhood size k is optimized within $\{2, 3, \dots, 10\}$ alongside the smoothing parameter $s \in \{0.5, 1, 2\}$, with Bayesian prior probabilities estimated through maximum likelihood on training data. RAKEL's configuration involves three-dimensional optimization: label subset size $k \in \{2, 3, 4, 5, 6\}$, ensemble size $m \in \{10, 20, 30, 40\}$, and decision threshold $\tau \in \{0.3, 0.4, \dots, 0.7\}$, where subsets are generated via random sampling without replacement. For BPMLL, neural architecture parameters are tuned through hidden layer ratios $\alpha \in \{0.5, 1.0\}$, learning rates $\eta \in \{0.1, 0.5\}$, regularization coefficients $\lambda \in \{0.01, 0.1\}$, and training epochs $T \in \{20, 50, 100\}$. The three main parameters of HOMI, β , λ , and γ , are selected from $\{1, 2, 5, 10\}$, $\{0.001, 0.01, 0.1, 1\}$ and $\{1, 1.5, 2, 2.5\}$

respectively to obtain the optimal combination, iteration count $iter = 100$, and scaling factor $s = 10$. For CLML, we perform a grid search over $\alpha, \beta, \lambda_1, \lambda_2 \in \{2^{-10}, 2^{-9}, \dots, 2^{10}\}$ with step size 2^2 . In WRAP, we follow the original literature and search over $\lambda \in \{0.1, 0.2, 0.3\}$ and $\alpha \in \{0.9, 0.8, 0.7\}$ with $d = \alpha \min(m, l)$ and step_size set to 1. For ILIA (BRKNN-ILIA), we use the polynomial kernel and fix the parameters as $\mu = 10^{-3}$, $\eta = 10^{-2}$, and $\gamma = 10^{-2}$. All kernel-based SVMs (RankSVM, MLTSVM, PMLTSVM) employ the Gaussian kernel function $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$, where the penalty parameter C and kernel width γ are jointly optimized over $\{2^{-5}, 2^{-4}, \dots, 2^5\}$ and $\{2^{-10}, 2^{-2}, \dots, 2^3\}$.

6.4. Discussion of the performance

We conducted extensive experiments on nine real-world multi-label datasets, evaluating eight baseline algorithms using five commonly adopted metrics. Among them, *Hamming Loss* and *F-score* mainly assess label prediction accuracy, while the remaining three metrics additionally reflect the label ranking capability of the models.

Tables 5- 9 report the mean and standard deviation of classification results obtained via 10-fold cross-validation. The best-performing result on each dataset is highlighted in bold. From the table, it is evident that our proposed PMLTSVM achieves competitive or superior performance across most datasets. In particular, when evaluating using the average rank across all metrics and datasets, PMLTSVM achieves the best overall performance, as illustrated in Figure 1. To further demonstrate the improvement in label ranking performance brought by the probabilistic formulation, we compare MLTSVM and PMLTSVM on the three ranking-sensitive metrics. As shown in Figure 2, PMLTSVM achieves superior ranking ability on nearly all datasets, with only a minor degradation on the *genbase* dataset. This exception may be attributed to the severe label imbalance in *genbase*, where the probability estimation in PMLTSVM might be slightly biased. In summary, the ranking capability of PMLTSVM is significantly improved, validating the effectiveness of our probabilistic ranking strategy. Moreover, PMLTSVM also demonstrates notable gains on other metrics compared to MLTSVM. This improvement is largely due to replacing the unstable thresholding mechanism in MLTSVM with a symmetric hyperplane decision based on probabilistic outputs, which enhances the robustness and overall predictive performance of the model.

6.5. Friedman tests, Holm's procedure and wilcoxon signed-ranks test

In this section, firstly, we employ the *Friedman test*[39] to conduct a statistical analysis of the relative performance among the compared algorithms. Let k, T and r_i^j denote the number of comparing approaches, the number of data sets and the rank of the j -th algorithm on the i -th data set respectively. The average rank of the j -th algorithm across all datasets is defined as $R_j = \frac{1}{T} \sum_{i=1}^T r_i^j$. Under the null hypothesis that all algorithms exhibit

“equal” performance, the following Friedman statistic F_F will be distributed according to F -distribution with $k - 1$ numerator degrees of freedom and $(k - 1)(T - 1)$ denominator degrees of freedom:

$$F_F = \frac{(T - 1)\chi_F^2}{T(k - 1) - \chi_F^2}, \quad (55)$$

where

$$\chi_F^2 = \frac{12T}{k(k + 1)} \left[\sum_{j=1}^k R_j^2 - \frac{k(k + 1)^2}{4} \right], \quad (56)$$

Table 2

Friedman test statistics over each evaluation metrics and the critical value at 0.05 significance level ($k = 10, T = 9$)

Evaluation Metric	F_F	Critical Value
Hammingloss	8.259	2.072
Rankloss	5.534	
AveragePrecision	4.261	
One-Error	4.338	

Table 2 summarizes the Friedman statistics F_F for each evaluation metric, along with the corresponding critical value at the 0.05 significance level (learning approaches $k = 10$, data sets $T = 9$). As shown in Table 2, the null hypothesis of equal performance among the comparing approaches is obviously rejected for all evaluation metrics.

To determine whether PMLTSVM exhibits significantly different performance from the other compared algorithms, we employ *Holm’s procedure*[39] as the post-hoc test by treating PMLTSVM as the control approach. Without loss of generality, PMLTSVM is denoted as the first comparing approach $\mathcal{A}_1$. For the other $k - 1$ comparing approaches $\mathcal{A}_j$ ($2 \leq j \leq k$), the one having the $(j - 1)$ -th largest average rank over all data sets is denoted as $\mathcal{A}_j$. Then, we have the test statistic for comparing $\mathcal{A}_1$ (i.e. PMLTSVM) and $\mathcal{A}_j$ as follows:

$$z_j = (R_1 - R_j) \sqrt{\frac{k(k + 1)}{6T}} \quad (2 \leq j \leq k), \quad (57)$$

Accordingly, we denote the p -value of z_j under the normal distribution as p_j . At a significance level α , Holm’s procedure operates by sequentially verifying whether $p_j < \alpha/(k - j + 1)$ holds in ascending order of j . Specifically, the procedure terminates at j^* , defined as the first index j for which $p_j < \alpha/(k - j + 1)$ does not hold. If $p_j < \alpha/(k - j + 1)$ is satisfied for all j , j^* is set to $k + 1$. Consequently, PMLTSVM is considered to exhibit significantly different performance from $\mathcal{A}_j$ for all $j \in \{2, \dots, j^* - 1\}$.

Table 3 reports the statistics by taking Holm’s procedure as the post-hoc test at 0.05 significance level, where PMLTSVM is treated as the control approach. From the Table 3, we can conclude that PMLTSVM significantly outperforms RAKEL in terms of all evaluation metrics except *hamming loss*, significantly outperforms CLML in terms of *hamming loss* and *rank-ing loss*, significantly outperforms HOMI and RankSVM in terms of *hamming loss* and *one-error*, significantly outperforms MLTSVM and BPMLL in terms of *hamming loss*, significantly outperforms ML-KNN and WRAP in terms of *one-error*.

To further validate the effectiveness and superiority of our proposed method, we additionally performed the Wilcoxon signed-rank test[39] to compare our method with MLTSVM and ILIA, respectively, across four evaluation metrics. A brief illustration of the Wilcoxon signed-rank test procedure is provided herein.

Let d_i denote the difference in performance scores between the two classifiers on the i -th of T datasets. These differences are ranked based on their absolute values; in the case of ties, average ranks are assigned. Let R^+ represent the sum of ranks for datasets where the second algorithm outperforms the first, and R^- represent the sum of ranks for the opposite scenario. For $d_i = 0$, their ranks are evenly split between R^+ and R^- ; if the count of such $d_i = 0$ is odd, one rank is discarded to maintain parity:

$$R^+ = \sum_{d_i > 0} \text{rank}(d_i) + \frac{1}{2} \sum_{d_i = 0} \text{rank}(d_i), \quad (58)$$

$$R^- = \sum_{d_i < 0} \text{rank}(d_i) + \frac{1}{2} \sum_{d_i = 0} \text{rank}(d_i) \quad (59)$$

Let S be the smaller of these two sums, $S = \min(R^+, R^-)$. Then the test statistic z is computed as:

$$z = \frac{S - \frac{1}{4}T(T + 1)}{\sqrt{\frac{1}{24}T(T + 1)(2T + 1)}} \quad (60)$$

Accordingly, we can obtain the p -value of z under the normal distribution for hypothesis testing.

Table 4

The p -values from the Wilcoxon signed-rank tests comparing PMLTSVM with MLTSVM and ILIA over each evaluation metrics at 0.05 significance level.

Evaluation Metric	p_M	p_I
Hammingloss	0.008	0.028
Rankloss	0.086	0.314
AveragePrecision	0.110	0.441
One-Error	0.161	0.110

Let p_M represent the p -values from the Wilcoxon signed-rank tests comparing PMLTSVM with MLTSVM, and p_I represent the p -values from the Wilcoxon signed-rank tests comparing PMLTSVM with ILIA. Table 4 reports the p -values from the Wilcoxon signed-rank tests comparing PMLTSVM with MLTSVM and ILIA over each evaluation metrics. Based on the Table 4, we can conclude that PMLTSVM significantly outperforms MLTSVM and ILIA in terms of hamming loss.

6.6. Parameters analysis

In this section, we investigate the impacts of parameters $c_1(c_2)$, γ , λ , and α on algorithm performance. Experiments are conducted on four randomly selected datasets: *Flags*, *Emotions*, *Scene*, and *Genbase*. By setting $c_1 = c_2$ and fixing λ and the probabilistic fitting parameter α , we analyze the effects of $c_1(c_2)$ and γ . The model demonstrates certain robustness, though excessive γ values degrade performance in some datasets (e.g., *Scene* and *Emotions*) due to overfitting, as shown in the four

Table 3

Comparison of PMLTSVM against other comparing approaches with *Holm's procedure* as the post-hoc test in terms of each evaluation metric (significance level $\alpha = 0.05$, comparing approaches $k = 10$).

Hamming loss					Ranking loss				
j	approach	z_j	p_j	$\alpha/(k-j+1)$	j	approach	z_j	p_j	$\alpha/(k-j+1)$
2	MLTSVM	-5.294	1.198e-7	0.006	2	RAkEL	-4.827	1.388e-6	0.006
3	CLML	-3.970	7.177e-5	0.006	3	CLML	-3.114	1.846e-3	0.006
4	HOMI	-3.425	6.139e-4	0.007	4	HOMI	-2.491	1.273e-2	0.007
5	BPMLL	-3.114	1.846e-3	0.008	5	WRAP	-2.180	2.927e-2	0.008
6	RankSVM	-2.647	8.123e-3	0.010	6	ML-KNN	-1.946	5.163e-2	0.010
7	WRAP	-2.180	2.927e-2	0.013	7	ILIA	-1.401	1.611e-1	0.013
8	RAkEL	-1.868	6.171e-2	0.017	8	MLTSVM	-1.090	2.758e-1	0.017
9	ILIA	-1.713	8.677e-2	0.025	9	BPMLL	-1.012	3.115e-1	0.025
10	ML-KNN	-1.090	2.758e-1	0.050	10	RankSVM	-1.012	3.115e-1	0.050

Average precision					One-error				
j	approach	z_j	p_j	$\alpha/(k-j+1)$	j	approach	z_j	p_j	$\alpha/(k-j+1)$
2	RAkEL	-4.671	2.997e-6	0.006	2	RAkEL	-4.282	1.854e-5	0.006
3	WRAP	-2.647	8.123e-3	0.006	3	RankSVM	-3.503	4.596e-4	0.006
4	ML-KNN	-2.491	1.273e-2	0.007	4	HOMI	-3.270	1.077e-3	0.007
5	HOMI	-2.258	2.397e-2	0.008	5	ML-KNN	-2.880	3.971e-3	0.008
6	RankSVM	-2.258	2.397e-2	0.010	6	WRAP	-2.725	6.435e-3	0.010
7	CLML	-1.946	5.163e-2	0.013	7	CLML	-2.335	1.952e-2	0.013
8	ILIA	-1.557	1.195e-1	0.017	8	ILIA	-2.335	1.952e-2	0.017
9	BPMLL	-1.012	3.115e-1	0.025	9	BPMLL	-1.479	1.391e-1	0.025
10	MLTSVM	-1.012	3.115e-1	0.050	10	MLTSVM	-0.856	3.918e-1	0.050

Table 5

Performance comparison of different methods with respect to *HammingLoss*. Bold indicates that the model performs best on this dataset.

Hammingloss ↓										
Approach	CAL500	birds	emotions	flags	genbase	image	medical	scene	yeast	
RAkEL	0.1545 ± 0.0066	0.0706 ± 0.0081	0.2173 ± 0.0254	0.2791 ± 0.0358	0.0029 ± 0.0034	0.2109 ± 0.0133	0.0250 ± 0.0023	0.1153 ± 0.0072	0.2064 ± 0.0148	
BPMLL	0.2342 ± 0.1593	0.2080 ± 0.0191	0.2234 ± 0.0237	0.2965 ± 0.0387	0.0135 ± 0.0035	0.1696 ± 0.0133	0.0200 ± 0.0016	0.0937 ± 0.0057	0.2096 ± 0.0102	
ML-KNN	0.1379 ± 0.0056	0.0497 ± 0.0057	0.2256 ± 0.0270	0.2810 ± 0.0450	0.0045 ± 0.0014	0.1699 ± 0.0078	0.0198 ± 0.0019	0.0877 ± 0.0086	0.1969 ± 0.0130	
HOMI	0.3737 ± 0.0725	0.0515 ± 0.0058	0.2279 ± 0.0306	0.2889 ± 0.0557	0.0074 ± 0.0030	0.2298 ± 0.0047	0.8788 ± 0.2808	0.1666 ± 0.0050	0.2108 ± 0.0094	
WRAP	0.1454 ± 0.0003	0.0742 ± 0.0024	0.2074 ± 0.0083	0.2962 ± 0.0032	0.0085 ± 0.0000	0.1875 ± 0.0016	0.0462 ± 0.0021	0.1102 ± 0.0003	0.1984 ± 0.0000	
CLML	0.1921 ± 0.0018	0.0620 ± 0.0054	0.3153 ± 0.0024	0.3104 ± 0.0184	0.0092 ± 0.0001	0.2432 ± 0.0054	0.0256 ± 0.0002	0.1797 ± 0.0056	0.3042 ± 0.0059	
ILIA	0.1434 ± 0.0002	0.0804 ± 0.0106	0.2103 ± 0.0043	0.3145 ± 0.0002	0.0125 ± 0.0002	0.1599 ± 0.0007	0.0112 ± 0.0022	0.0842 ± 0.0045	0.1924 ± 0.0012	
RankSVM	0.1606 ± 0.0052	0.0783 ± 0.0067	0.2831 ± 0.0357	0.3161 ± 0.0302	0.0116 ± 0.0025	0.1572 ± 0.0095	0.0275 ± 0.0024	0.0835 ± 0.0082	0.1968 ± 0.0113	
MLTSVM	0.8462 ± 0.0084	0.1680 ± 0.0605	0.2834 ± 0.0413	0.3973 ± 0.0446	0.0193 ± 0.0091	0.2610 ± 0.0304	0.1723 ± 0.0011	0.1489 ± 0.0270	0.5985 ± 0.0521	
PMLTSVM	0.1375 ± 0.0049	0.0464 ± 0.0065	0.2082 ± 0.0247	0.2965 ± 0.0504	0.0034 ± 0.0013	0.1563 ± 0.0162	0.0163 ± 0.0020	0.0731 ± 0.0063	0.1892 ± 0.0098	

Table 6

Performance comparison of different methods with respect to *Rankloss*. Bold indicates that the model performs best on this dataset.

Rankingloss ↓										
Approach	CAL500	birds	emotions	flags	genbase	image	medical	scene	yeast	
RAkEL	0.9221 ± 0.0560	0.2262 ± 0.0388	0.2706 ± 0.0440	0.3260 ± 0.0505	0.0542 ± 0.0830	0.3307 ± 0.0337	0.2326 ± 0.0738	0.2162 ± 0.0186	0.3519 ± 0.0404	
BPMLL	0.1827 ± 0.0089	0.0953 ± 0.0169	0.1662 ± 0.0207	0.2230 ± 0.0430	0.0019 ± 0.0034	0.1598 ± 0.0172	0.0186 ± 0.0073	0.0699 ± 0.0075	0.1760 ± 0.0145	
ML-KNN	0.1816 ± 0.0077	0.1002 ± 0.0210	0.1838 ± 0.0264	0.2181 ± 0.0405	0.0056 ± 0.0065	0.1751 ± 0.0144	0.0432 ± 0.0106	0.0785 ± 0.0120	0.1720 ± 0.0168	
HOMI	0.2699 ± 0.0348	0.0971 ± 0.0114	0.1861 ± 0.0341	0.2158 ± 0.0448	0.0042 ± 0.0057	0.1860 ± 0.0172	0.0817 ± 0.0500	0.0882 ± 0.0095	0.1773 ± 0.0162	
WRAP	0.1892 ± 0.0094	0.0992 ± 0.0045	0.1572 ± 0.0062	0.2678 ± 0.0104	0.0623 ± 0.0000	0.1564 ± 0.0028	0.0326 ± 0.0034	0.1030 ± 0.0064	0.1702 ± 0.0004	
CLML	0.1823 ± 0.0132	0.0998 ± 0.0279	0.1974 ± 0.0123	0.2264 ± 0.0194	0.0432 ± 0.0002	0.1502 ± 0.0023	0.0942 ± 0.0032	0.1114 ± 0.0088	0.3566 ± 0.0012	
ILIA	0.1809 ± 0.0124	0.0944 ± 0.0021	0.1792 ± 0.0023	0.2002 ± 0.0011	0.0684 ± 0.0012	0.1599 ± 0.0020	0.0402 ± 0.0186	0.0952 ± 0.0013	0.1698 ± 0.0027	
RankSVM	0.1802 ± 0.0088	0.1115 ± 0.0160	0.1786 ± 0.0349	0.2181 ± 0.0329	0.0067 ± 0.0061	0.1506 ± 0.0125	0.0690 ± 0.0146	0.0634 ± 0.0105	0.1620 ± 0.0146	
MLTSVM	0.1823 ± 0.0100	0.0962 ± 0.0132	0.1711 ± 0.0366	0.2330 ± 0.0447	0.0012 ± 0.0017	0.1517 ± 0.0081	0.1426 ± 0.0081	0.0626 ± 0.0123	0.1576 ± 0.0152	
PMLTSVM	0.1802 ± 0.0081	0.0923 ± 0.0176	0.1691 ± 0.0311	0.2218 ± 0.0339	0.0131 ± 0.0086	0.1455 ± 0.0156	0.0755 ± 0.0123	0.0606 ± 0.0072	0.1571 ± 0.0129	

Table 7

Performance comparison of different methods with respect to *AveragePrecision*. Bold indicates that the model performs best on this dataset.

Averageprecision ↑										
Approach	CAL500	birds	emotions	flags	genbase	image	medical	scene	yeast	
RAkEL	0.2444 ± 0.0267	0.5394 ± 0.0446	0.7723 ± 0.0289	0.7888 ± 0.0311	0.9615 ± 0.0769	0.7383 ± 0.0160	0.7354 ± 0.0563	0.7971 ± 0.0213	0.7404 ± 0.0205	
BPMLL	0.4993 ± 0.0118	0.6203 ± 0.0405	0.7887 ± 0.0275	0.8064 ± 0.0262	0.9966 ± 0.0051	0.8068 ± 0.0201	0.8579 ± 0.0174	0.8717 ± 0.0130	0.7518 ± 0.0172	
ML-KNN	0.4955 ± 0.0103	0.5733 ± 0.0586	0.7806 ± 0.0381	0.8029 ± 0.0294	0.9865 ± 0.0086	0.7954 ± 0.0154	0.7669 ± 0.0169	0.8635 ± 0.0165	0.7600 ± 0.0217	
HOMI	0.4484 ± 0.0202	0.6102 ± 0.0335	0.7860 ± 0.0321	0.8034 ± 0.0258	0.9959 ± 0.0060	0.7634 ± 0.0151	0.8884 ± 0.0217	0.8378 ± 0.0182	0.7529 ± 0.0167	
WRAP	0.4804 ± 0.0074	0.6239 ± 0.0374	0.7887 ± 0.0403	0.7265 ± 0.0076	0.9432 ± 0.0872	0.8257 ± 0.0190	0.7498 ± 0.0356	0.8899 ± 0.0034	0.1589 ± 0.0374	
CLML	0.4882 ± 0.0024	0.6200 ± 0.0503	0.7902 ± 0.0653	0.8094 ± 0.0881	0.9653 ± 0.0032	0.8127 ± 0.0062	0.7462 ± 0.0452	0.8894 ± 0.0165	0.1621 ± 0.0012	
ILIA	0.5164 ± 0.0193	0.5903 ± 0.0933	0.7894 ± 0.0083	0.8005 ± 0.0021	0.9842 ± 0.0023	0.8267 ± 0.0173	0.7784 ± 0.0183	0.8798 ± 0.0231	0.1603 ± 0.0234	
RankSVM	0.5007 ± 0.0146	0.5535 ± 0.0393	0.7783 ± 0.0429	0.8050 ± 0.0346	0.9796 ± 0.0103	0.8201 ± 0.0108	0.6464 ± 0.0377	0.8779 ± 0.0143	0.7613 ± 0.0191	
MLTSVM	0.4977 ± 0.0124	0.6204 ± 0.0508	0.7939 ± 0.0415	0.8014 ± 0.0415	0.9957 ± 0.0062	0.8193 ± 0.0130	0.7433 ± 0.0212	0.8907 ± 0.0178	0.7732 ± 0.0193	
PMLTSVM	0.4982 ± 0.0110	0.6335 ± 0.0522	0.7968 ± 0.0400	0.8060 ± 0.0303	0.9778 ± 0.0119	0.8302 ± 0.0168	0.7565 ± 0.0339	0.8928 ± 0.0106	0.7769 ± 0.0143	

Table 8

Performance comparison of different methods with respect to *One-Error*. Bold indicates that the model performs best on this dataset.

Approach	One - error ↓									
	CAL500	birds	emotions	flags	genbase	image	medical	scene	yeast	
RaKEL	0.5038 ± 0.0883	0.7163 ± 0.0611	0.3187 ± 0.0543	0.2532 ± 0.0806	0.0348 ± 0.0761	0.3830 ± 0.0257	0.3080 ± 0.0619	0.2983 ± 0.0317	0.2958 ± 0.0392	
BPMLL	0.1176 ± 0.0649	0.6900 ± 0.0524	0.3097 ± 0.0420	0.2013 ± 0.0492	0.0002 ± 0.0000	0.3005 ± 0.0366	0.2136 ± 0.0243	0.2194 ± 0.0232	0.2350 ± 0.0272	
ML-KNN	0.1156 ± 0.0675	0.7288 ± 0.0632	0.3014 ± 0.0707	0.2529 ± 0.0691	0.0151 ± 0.0117	0.3105 ± 0.0246	0.3241 ± 0.0233	0.2302 ± 0.0275	0.2325 ± 0.0336	
HOMI	0.1172 ± 0.0481	0.6915 ± 0.0628	0.3215 ± 0.0672	0.2276 ± 0.0649	0.0060 ± 0.0099	0.3595 ± 0.0406	0.6538 ± 0.0596	0.2584 ± 0.0175	0.2516 ± 0.0285	
WRAP	0.1750 ± 0.0012	0.6730 ± 0.0783	0.2652 ± 0.0073	0.2367 ± 0.0187	0.0456 ± 0.0003	0.2674 ± 0.0035	0.3104 ± 0.0110	0.2764 ± 0.0083	0.2996 ± 0.0081	
CLML	0.1169 ± 0.0021	0.7213 ± 0.0234	0.2862 ± 0.0071	0.2123 ± 0.0336	0.0023 ± 0.0000	0.2698 ± 0.0012	0.3083 ± 0.0007	0.2997 ± 0.0215	0.3876 ± 0.0021	
ILIA	0.1174 ± 0.0045	0.6899 ± 0.0122	0.2993 ± 0.0088	0.2907 ± 0.0431	0.0135 ± 0.0054	0.3102 ± 0.0765	0.2387 ± 0.0387	0.2103 ± 0.0109	0.2586 ± 0.0454	
RankSVM	0.1196 ± 0.0652	0.7366 ± 0.0449	0.3216 ± 0.0678	0.2279 ± 0.0933	0.0302 ± 0.0166	0.2750 ± 0.0184	0.4969 ± 0.0541	0.2135 ± 0.0214	0.2404 ± 0.0302	
MLTSVM	0.1156 ± 0.0675	0.6838 ± 0.0618	0.2778 ± 0.0615	0.2271 ± 0.1070	0.0030 ± 0.0091	0.2750 ± 0.0286	0.3150 ± 0.0310	0.1832 ± 0.0278	0.2238 ± 0.0278	
PMLTSVM	0.1156 ± 0.0346	0.6729 ± 0.0639	0.2846 ± 0.0802	0.1961 ± 0.0840	0.0152 ± 0.0179	0.2520 ± 0.0265	0.3017 ± 0.0385	0.1795 ± 0.0161	0.2214 ± 0.0181	

Table 9

Performance comparison of different methods on Delicious and Corel16k1 datasets with respect to four metrics: Hamming Loss (HL), One-Error (OE), Ranking Loss (RL), and Average Precision (AP). Bold indicates the best result for each metric and dataset.

Approach	Delicious				Corel16k1			
	HL ↓	OE ↓	RL ↓	AP ↑	HL ↓	OE ↓	RL ↓	AP ↑
HOMI	0.0184±0.0003	0.5763±0.0020	0.1731±0.0039	0.3648±0.0064	0.0179±0.0003	0.6507±0.0091	0.1552±0.0027	0.3448±0.0009
WRAP	0.0191±0.0004	0.3369±0.0128	0.1273±0.0026	0.3672±0.0060	0.0191±0.0004	0.6318±0.0052	0.1453±0.0010	0.3442±0.0022
CLML	0.0294±0.0012	0.3862±0.0006	0.1350±0.0174	0.3701±0.0009	0.0193±0.0012	0.6948±0.0031	0.2843±0.0050	0.4021±0.0017
ILIA	0.0185±0.0063	0.3269±0.0038	0.1503±0.0021	0.3892±0.0102	0.0204±0.0001	0.7949±0.0068	0.1895±0.0012	0.2472±0.0018
PMLTSVM	0.0182±0.0001	0.3267±0.0054	0.1426±0.0011	0.3908±0.0013	0.0183±0.0004	0.6215±0.0074	0.1398±0.0008	0.3763±0.0073

subfigures of Fig. 3. By fixing $c_1(c_2)$ and γ , we further examine the influences of λ and α . Both parameters exhibit similar trends: Excessively large λ values amplify the eigenvalues of the quadratic problem’s matrix, significantly complicating matrix inversion. These results are presented in Fig. 4. In conclusion, appropriate parameter selection is critical for optimal model performance.

6.7. Safe screening rule

The screening effectiveness of our proposed SSR- $c_1(c_2)$ and SSR- λ_k methods is systematically demonstrated in Tables 10 and 11. Three key metrics are defined to quantitatively assess SSR performance: *Ratio* denotes the proportion of identified 0/1 components in dual variables through SSR screening; *Speedup* represents the computational acceleration ratio defined as $T_{\text{PMLTSVM}}/T_{\text{SSR+PMLTSVM}}$; *Accuracy* reflects the mean prediction accuracy measured by label-wise classification. All experiments were conducted with Parameter interval $d = 1 \times 10^{-5}$, distinguishing known and unknown parameters, using the CVXOPT solver with embedded SSR through 5-fold cross-validation. Experimental results reveal three critical findings: First, identical prediction accuracy between SSR-enhanced and baseline methods across all parameter configurations confirms SSR’s safety in preserving classification boundaries. This is consistent with our theoretical proof in Section 4. Since the dual problem of PMLTSVM is convex and our acceleration method is derived from rigorous convex analysis, SSR can safely filter out certain redundant samples before training, and thus it naturally does not affect the prediction accuracy. Second, universal training time reduction shows SSR- λ_k achieves 2.72 – 49.30 speedup versus SSR- $c_1(c_2)$ ’s 3.31 – 32.12 improvement, attributable to tighter inequality constraints in the $c_1(c_2)$ formulation. Third, parameter-dependent acceleration characteristics in Figures 5 and 6 demonstrate maximum speedup during dense parameter sampling intervals, maintaining about 3-10× acceleration even at extreme parameter values. This compre-

hensive evidence establishes SSR as a theoretically grounded acceleration strategy that guarantees computational efficiency without compromising classification safety. The method’s parameter adaptiveness particularly recommends it for applications requiring intensive hyperparameter optimization.

6.8. Ablation Study

We conduct an ablation study on three benchmark datasets: *Scene*, *Image*, and *Delicious*. We evaluate the effect of each module in our framework: dual planes (DP), probabilistic output optimization (POO), and safe screening rule (SSR). The SSR module is specifically designed for the dual-plane framework, while the POO module requires the dual-plane outputs to compute probabilities. Therefore, we consider the following five configurations: (1) all three modules enabled (ours), (2) DP+POO without SSR, (3) DP+SSR without POO, (4) DP only, and (5) baseline MLTSVM without any additional modules. We report One-Error, Hamming Loss, Ranking Loss, Average Precision, and training *Time* in seconds for each dataset, and the results are summarized in Tables 12–14. The experimental results corroborate our theoretical analysis in Section 4, demonstrating that SSR serves as a safe acceleration technique that does not compromise solution accuracy, while DP and POO effectively enhance the overall performance of the model.

7. Conclusion

This paper addresses the problem of multi-label learning and proposes a novel approach named PMLTSVM. PMLTSVM reconsiders the threshold mechanism and ranking method of the traditional MLTSVM, which effectively enhances model performance. Additionally, the sparsity of the model is leveraged to construct a Safe Screening Rule (SSR), significantly accelerating training and parameter selection. In the future, exploring how to extend this method to online and semi-supervised learning is a promising direction. We also acknowledge certain

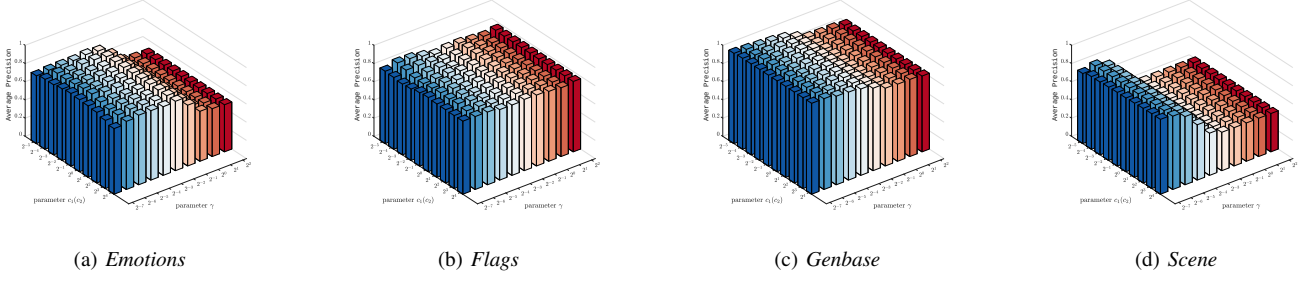


Fig. 3. Sensitivity analysis of parameters $c_1 = c_2 = c, \gamma$ on four datasets.

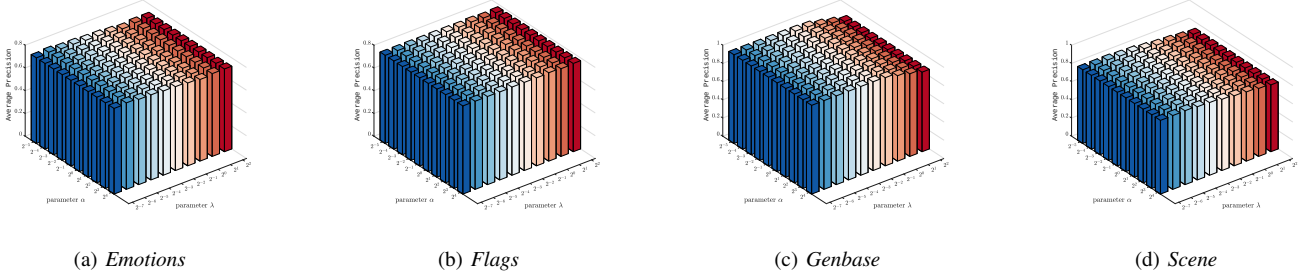


Fig. 4. Sensitivity analysis of parameters λ, α_p on four datasets.

Table 10
Performance and Time comparison with and without SSR- $c_1(c_2)$ across datasets.

Datasets	PMLTSVM		SSR- $c_1(c_2)$ +PMLTSVM		Speedup	Ratio
	Accuracy(%)	Time(s)	Accuracy(%)	Time(s)		
Scene	90.2800 $\pm$ 0.6000	15.0868 $\pm$ 0.6572	90.2800 $\pm$ 0.6000	2.3339 $\pm$ 0.1486	6.47	0.5097 $\pm$ 0.0027
Yeast	80.2500 $\pm$ 0.5200	32.7705 $\pm$ 1.0935	80.2500 $\pm$ 0.5200	7.0575 $\pm$ 0.2647	4.64	0.4494 $\pm$ 0.0017
CAL500	83.3600 $\pm$ 0.3200	14.7972 $\pm$ 2.7232	83.3600 $\pm$ 0.3200	4.4672 $\pm$ 1.0402	3.31	0.6606 $\pm$ 0.0035
Birds	95.1800 $\pm$ 0.3200	2.1490 $\pm$ 0.0311	95.1800 $\pm$ 0.3200	0.5880 $\pm$ 0.0182	3.66	0.6860 $\pm$ 0.0087
Emotions	79.6000 $\pm$ 0.9900	0.4948 $\pm$ 0.0156	79.6000 $\pm$ 0.9900	0.1148 $\pm$ 0.0193	4.31	0.8189 $\pm$ 0.1176
Flags	73.5700 $\pm$ 1.7800	0.1092 $\pm$ 0.0146	73.5700 $\pm$ 1.7800	0.0034 $\pm$ 0.0002	32.12	0.7842 $\pm$ 0.0093
Image	84.6000 $\pm$ 0.9700	7.1482 $\pm$ 0.2348	84.6000 $\pm$ 0.9700	1.1315 $\pm$ 0.0548	6.32	0.4769 $\pm$ 0.0086
Medical	97.6200 $\pm$ 0.0600	14.5511 $\pm$ 0.8862	97.6200 $\pm$ 0.0600	4.2420 $\pm$ 0.3083	3.43	0.2945 $\pm$ 0.0091
Genbase	99.7500 $\pm$ 0.1500	3.1252 $\pm$ 0.0652	99.7500 $\pm$ 0.1500	0.6769 $\pm$ 0.0289	4.62	0.7853 $\pm$ 0.0037

Table 11
Performance and Time comparison with and without SSR- λ_k across datasets.

Datasets	PMLTSVM		SSR- λ_k +PMLTSVM		Speedup	Ratio
	Accuracy(%)	Time(s)	Accuracy(%)	Time(s)		
Scene	92.5200 $\pm$ 0.4600	15.2944 $\pm$ 0.1780	92.5200 $\pm$ 0.4600	2.3254 $\pm$ 0.0354	6.58	0.8946 $\pm$ 0.0020
Yeast	80.6000 $\pm$ 0.5300	36.4726 $\pm$ 0.2968	80.6000 $\pm$ 0.5300	7.2344 $\pm$ 0.0864	5.04	0.7787 $\pm$ 0.0048
CAL500	86.3200 $\pm$ 0.1600	12.2524 $\pm$ 0.0998	86.3200 $\pm$ 0.1600	3.3151 $\pm$ 0.0859	3.70	0.7720 $\pm$ 0.0037
Birds	95.5400 $\pm$ 0.5400	2.9046 $\pm$ 0.4472	95.5400 $\pm$ 0.5400	0.7390 $\pm$ 0.1165	3.93	0.8272 $\pm$ 0.0046
Emotions	77.9200 $\pm$ 1.5600	0.5439 $\pm$ 0.0126	77.9200 $\pm$ 1.5600	0.1646 $\pm$ 0.0072	3.30	0.9323 $\pm$ 0.0114
Flags	73.7000 $\pm$ 1.2700	0.1134 $\pm$ 0.0093	73.7000 $\pm$ 1.2700	0.0023 $\pm$ 0.0029	49.30	0.9087 $\pm$ 0.0031
Image	84.8800 $\pm$ 0.7800	7.4025 $\pm$ 0.0936	84.8800 $\pm$ 0.7800	1.9344 $\pm$ 0.0339	3.83	0.8446 $\pm$ 0.0023
Medical	97.9500 $\pm$ 0.0900	14.6701 $\pm$ 1.5867	97.9500 $\pm$ 0.0900	5.3835 $\pm$ 0.8269	2.72	0.4287 $\pm$ 0.0278
Genbase	99.8000 $\pm$ 0.0500	3.1252 $\pm$ 0.0652	99.8000 $\pm$ 0.0500	0.1769 $\pm$ 0.0289	17.67	0.8863 $\pm$ 0.0022

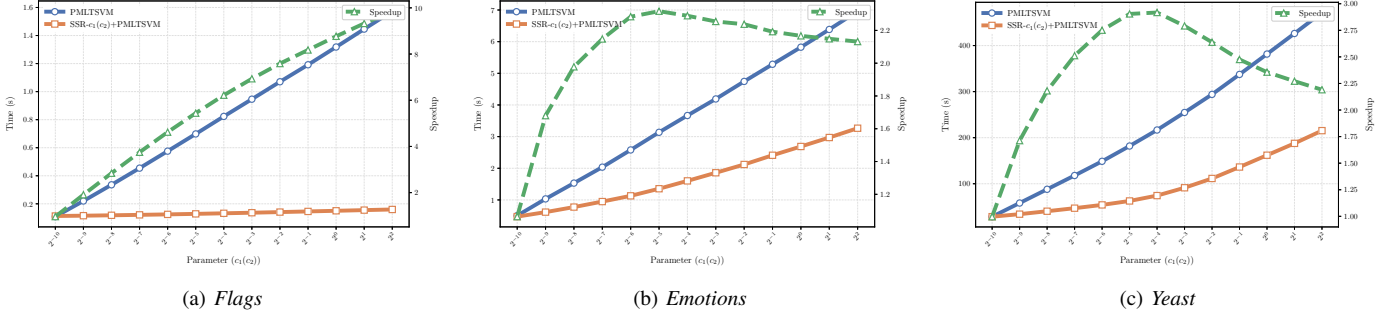


Fig. 5. Comparison of parameter tuning time between using and not using SSR- $c_1(c_2)$

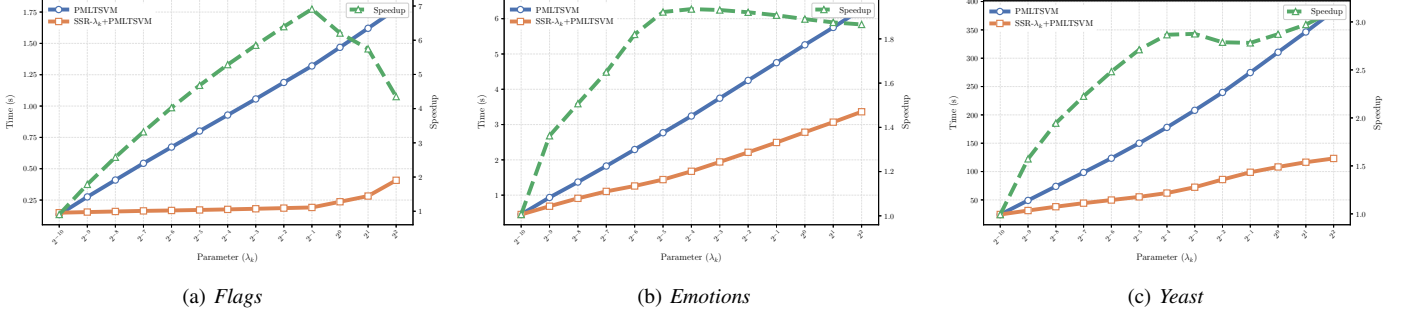


Fig. 6. Comparison of parameter tuning time between using and not using SSR- λ_k

Table 12

Ablation study on the *Scene* dataset.

Method	DP	POO	SSR	One-Error ↓	Hamming Loss ↓	Ranking Loss ↓	Average Precision ↑	Time (s) ↓
Ours	✓	✓	✓	0.1795	0.0731	0.0606	0.8928	2.33
DP + POO (no SSR)	✓	✓	✗	0.1795	0.0731	0.0606	0.8928	15.29
DP + SSR (no POO)	✓	✗	✓	0.1812	0.1244	0.6110	0.8913	2.24
DP only	✓	✗	✗	0.1812	0.1244	0.6110	0.8913	14.42
Baseline (MLTSVM)	✗	✗	✗	0.1834	0.1489	0.0626	0.8907	9.84

Table 13

Ablation study on the *Image* dataset.

Method	DP	POO	SSR	One-Error ↓	Hamming Loss ↓	Ranking Loss ↓	Average Precision ↑	Time (s) ↓
Ours	✓	✓	✓	0.2520	0.1563	0.1455	0.8302	1.13
DP + POO (no SSR)	✓	✓	✗	0.2520	0.1563	0.1455	0.8302	7.15
DP + SSR (no POO)	✓	✗	✓	0.2612	0.2364	0.1502	0.8294	1.05
DP only	✓	✗	✗	0.2612	0.2364	0.1502	0.8294	7.02
Baseline (MLTSVM)	✗	✗	✗	0.2750	0.2610	0.1517	0.8193	4.89

Table 14

Ablation study on the *Delicious* dataset.

Method	DP	POO	SSR	One-Error ↓	Hamming Loss ↓	Ranking Loss ↓	Average Precision ↑	Time (s) ↓
Ours	✓	✓	✓	0.3267	0.0182	0.1426	0.3908	257.4
DP + POO (no SSR)	✓	✓	✗	0.3267	0.0182	0.1426	0.3908	1027.8
DP + SSR (no POO)	✓	✗	✓	0.3401	0.0210	0.1625	0.3892	242.4
DP only	✓	✗	✗	0.3401	0.0210	0.1625	0.3892	987.8
Baseline (MLTSVM)	✗	✗	✗	0.3897	0.0239	0.1784	0.3791	663.9

limitations of our work: the training process is not particularly GPU-friendly, suggesting that developing parallelizable optimization strategies (e.g., via CUDA) could further improve efficiency; and the exploitation of higher-order label relationships remains an open area for improvement, where recent graph-based approaches may offer valuable insights. Moreover, inspired by recent studies such as Jiang[43] and Duan[44], integrating multi-view learning with multi-label classification of-

fers an exciting avenue for leveraging complementary information to enhance performance, making it another worthwhile direction for future exploration.

Acknowledgments

The authors gratefully acknowledge the helpful comments and suggestions of the reviewers, which have improved the presentation. This work was supported by the National Natural

Science Foundation of China (No.12071475, 11671010) and the Beijing Students' innovation and entrepreneurship training program.

References

- [1] W. Liu, H. Wang, X. Shen, I. W. Tsang, The emerging trends of multi-label learning, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44 (11) (2022) 7955–7974.
- [2] H. Wang, W. Guan, C. Jianpeng, Z. Wang, D. Zhou, Towards heterogeneous long-tailed learning: Benchmarking, metrics, and toolbox, *Advances in Neural Information Processing Systems* 37 (2024) 73098–73123.
- [3] Z. Xiang, S. Cui, C. Shang, J. Jiang, L. Zhang, Gmod: Graph-driven momentum distillation framework with active perception of disease severity for radiology report generation, in: *Medical Image Computing and Computer Assisted Intervention – MICCAI 2024*, 2024, pp. 295–305.
- [4] H. D. Nguyen, X. S. Vu, D. T. Le, I. Assoc Advancement Artificial, Modular graph transformer networks for multi-label image classification, in: *35th AAAI Conference on Artificial Intelligence/33rd Conference on Innovative Applications of Artificial Intelligence/11th Symposium on Educational Advances in Artificial Intelligence*, Vol. 35, 2021, pp. 9092–9100.
- [5] M. Zhang, Z. Zhou, A review on multi-label learning algorithms, *IEEE Transactions on Knowledge & Data Engineering* 26 (8) (2014) 1819–1837.
- [6] M. R. Boutell, J. Luo, X. Shen, C. M. Brown, Learning multi-label scene classification, *Pattern Recognition* 37 (9) (2004) 1757–1771.
- [7] R. Wang, S. Ye, K. Li, S. Kwong, Bayesian network based label correlation analysis for multi-label classifier chain, *Information Sciences* 554 (2021) 256–275.
- [8] J. Frnkranz, E. Hllrmeier, E. LozaMencla, K. Brinker, Multilabel classification via calibrated label ranking, *Machine Learning* 73 (2) (2008) 133–153.
- [9] G. Tsoumakas, I. Vlahavas, Random k-labelsets: An ensemble method for multilabel classification, in: *Machine Learning: ECML 2007*, 2007, pp. 406–417.
- [10] M. L. Zhang, Z. H. Zhou, MI-knn: A lazy learning approach to multi-label learning, *Pattern Recognition* 40 (7) (2007) 2038–2048.
- [11] P. Li, H. Li, M. Wu, Multi-label ensemble based on variable pairwise constraint projection, *Information Sciences* 222 (2013) 269–281.
- [12] N. Ghamrawi, A. McCallum, Collective multi-label classification, in: *Proceedings of the 14th ACM International Conference on Information and Knowledge Management*, 2005, p. 195–200.
- [13] A. Elisseeff, J. Weston, A kernel method for multi-labelled classification, in: *Advances in Neural Information Processing Systems*, Vol. 14, 2001, pp. 681–687.
- [14] M. Xu, Y.-F. Li, Z.-H. Zhou, Robust multi-label learning with pro loss, *IEEE Transactions on Knowledge and Data Engineering* 32 (8) (2019) 1610–1624.
- [15] Y. Zhang, B. Kang, B. Hooi, S. Yan, J. Feng, Deep long-tailed learning: A survey, *IEEE transactions on pattern analysis and machine intelligence* 45 (9) (2023) 10795–10816.
- [16] X. Wang, Y. Yang, Y. Xu, Q. Chen, H. Wang, H. Gao, Predicting hypoglycemic drugs of type 2 diabetes based on weighted rank support vector machine, *Knowledge-Based Systems* 197 (2020) 105868.
- [17] Jayadeva, R. Khemchandani, S. Chandra, Twin support vector machines for pattern classification, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 29 (5) (2007) 905–910.
- [18] B. Richharia, M. Tanveer, Eeg signal classification using universum support vector machine, *Expert Systems with Applications* 106 (2018) 169–182.
- [19] M. Tanveer, T. Rajani, R. Rastogi, Y.-H. Shao, M. Ganaie, Comprehensive review on twin support vector machines, *Annals of Operations Research* 339 (3) (2024) 1223–1268.
- [20] R. Sharma, T. Goel, M. Tanveer, M. Al-Dhaifallah, Alzheimer's disease diagnosis using ensemble of random weighted features and fuzzy least square twin support vector machine, *IEEE Transactions on Emerging Topics in Computational Intelligence* 9 (2) (2025) 1281–1291.
- [21] W. J. Chen, Y. H. Shao, C. N. Li, N. Y. Deng, Mltsvm: A novel twin support vector machine to multi-label learning, *Pattern Recognition* 52 (2016) 61–74.
- [22] Y. H. Shao, N. Y. Deng, Z. M. Yang, W. J. Chen, Z. Wang, Probabilistic outputs for twin support vector machines, *Knowledge-Based Systems* 33 (2012) 145–151.
- [23] I. Arrieta-Ibarra, P. Gujral, J. Tannen, M. Tygert, C. Xu, Metrics of calibration for probabilistic predictions, *Journal of Machine Learning Research* 23 (351) (2022) 1–54.
- [24] P. H. Chen, R. E. Fan, C. J. Lin, A study on smo-type decomposition methods for support vector machines, *IEEE Trans Neural Netw* 17 (4) (2006) 893–908.
- [25] C.-J. Hsieh, K.-W. Chang, C.-J. Lin, S. S. Keerthi, S. Sundararajan, A dual coordinate descent method for large-scale linear svm, in: *Proceedings of the 25th International Conference on Machine Learning*, 2008, p. 408–415.
- [26] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, S. Boyd, OSQP: an operator splitting solver for quadratic programs, *Mathematical Programming Computation* 12 (4) (2020) 637–672.
- [27] L. E. Ghaoui, V. Viallon, T. Rabbani, Safe feature elimination in sparse supervised learning, *Pacific Journal of Optimization* 8 (4) (2010) 667–698.
- [28] J. Wang, P. Wonka, J. Ye, Scaling svm and least absolute deviations via exact data reduction, in: *Proceedings of the 31st International Conference on Machine Learning*, Vol. 32, 2014, pp. 523–531.
- [29] M. Ma, Y. Xu, Multi-parameter safe screening rule for hinge-optimal margin distribution machine, *Applied Intelligence* 49 (3) (2020) 1–12.
- [30] X. Pang, J. Zhao, Y. Xu, A novel ramp loss-based multi-task twin support vector machine with multi-parameter safe acceleration, *Neural Networks* 150 (2022) 194–212.
- [31] Z. Yang, W. Chen, H. Zhang, Y. Xu, L. Shi, J. Zhao, A safe screening rule with bi-level optimization of ν support vector machine, *Pattern Recognition* 155 (2024) 110644.
- [32] X. Wang, Y. Xu, Label pair of instances-based safe screening for multilabel rank support vector machine, *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 53 (3) (2023) 1907–1919.
- [33] Y.-H. Shao, C.-H. Zhang, X.-B. Wang, N.-Y. Deng, Improvements on twin support vector machines, *IEEE Transactions on Neural Networks* 22 (6) (2011) 962–968.
- [34] Y. Zhang, Y. Xu, C. Xu, P. Zhong, Safe instance screening for primal multi-label prosvm, *Knowledge-Based Systems* 229 (2021) 107362.
- [35] H. T. Lin, C. J. Lin, R. C. Weng, A note on platt's probabilistic outputs for support vector machines, *Machine Learning* 68 (3) (2007) 267–276.
- [36] M. L. Zhang, Z. H. Zhou, Multilabel neural networks with applications to functional genomics and text categorization, *IEEE Transactions on Knowledge & Data Engineering* 18 (10) (2006) 1338–1351.
- [37] J. Read, B. Pfahringer, G. Holmes, E. Frank, Classifier chains for multi-label classification, *Machine Learning* 85 (3) (2011) 333–359.
- [38] C. Si, Y. Jia, R. Wang, M.-L. Zhang, Y. Feng, C. Qu, Multi-label classification with high-rank and high-order label correlations, *IEEE Transactions on Knowledge and Data Engineering* 36 (8) (2023) 4076–4088.
- [39] J. Demšar, Statistical comparisons of classifiers over multiple data sets, *J. Mach. Learn. Res.* 7 (2006) 1–30.
- [40] Z.-B. Yu, M.-L. Zhang, Multi-label classification with label-specific feature generation: A wrapped approach, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44 (9) (2022) 5199–5210.
- [41] J. Li, P. Li, X. Hu, K. Yu, Learning common and label-specific features for multi-label classification with correlation information, *Pattern Recognition* 121 (2022) 108259.
- [42] J.-X. Mao, Y. Rui, M.-L. Zhang, Implicit relative labeling-importance aware multi-label metric learning, *Proceedings of the AAAI Conference on Artificial Intelligence* 39 (18) (2025) 19414–19422.
- [43] Y. Jiang, L. Zhao, Z. Jiang, Y. Ye, X. Lu, Semantic consistency enhancement and contribution-driven network for partial multi-view incomplete multi-label classification, *Journal of King Saud University Computer and Information Sciences* 37 (4) (2025) 53.

- [44] K. Duan, S. Cui, H. Shinnou, S. Bao, Learning consistent representation for incomplete multi-view weak multi-label classification, *Neurocomputing* (2025) 131238.



Bohan Zhang is currently pursuing for the bachelor's degree in Artificial Intelligence in the College of Information and Electrical Engineering, China Agricultural University, Beijing, China. He is interested in machine learning, deep learning and data mining.



Xincheng Lv is currently pursuing for the bachelor's degree in Mathematics and Applied Mathematics in the College of Science, China Agricultural University, Beijing, China. He is interested in machine learning, deep learning and data mining.



Chang Xu received the B.S. degree from the School of Information Management, Beijing Information Science and Technology University, Beijing, China, in 2020, and the M.S. degree from College of Information and Electrical Engineering, China Agricultural University, Beijing, China, in 2022. Now, she is currently pursuing the Ph.D. degree in College of Information and Electrical Engineering, China Agricultural University, Beijing, China.

Her current research interests include Data Mining and Agricultural information processing. Her research has appeared in *Applied Soft Computing*, *Neural Networks*, *Journal of Industrial Information Integration*, *Knowledge-Based Systems*, *Computers and Electronics in Agriculture*, and so on.



Yitian Xu received the Ph.D. degree from the College of Science, China Agricultural University, Beijing, China, in 2007. He is currently a Professor at the College of Science, China Agricultural University.

He has authored 110+ papers. His current research interests include machine learning and data mining. Prof. Xu's research has appeared in *IEEE TPAMI*, *IEEE TNNLS*, *IEEE TCYB*, *IEEE TSP*, *IEEE TSMC: Systems*, *ICML*, and so on.